

Workforce Management Platform

Technical System Documentation

MVP 1

Project purpose: A production-oriented workforce management platform covering employees, departments, scheduling, attendance, working hours, overtime, leave, labor cost, reporting, and audit logging for a single organization at a time (multi-tenant at the data layer).

Technology stack: Python 3.14, Flask 3.1, SQLAlchemy 2.0, Flask-Migrate (Alembic), PostgreSQL, Flask-Login, Flask-WTF, argon2-cffi, Gunicorn, server-rendered Jinja templates, plain CSS and vanilla JavaScript.

Documentation version: 1.0

Generated: August 2026

This document is generated directly from the project's source code, database migrations, and test suite. It contains no fictional companies, infrastructure, or functionality. Any feature that is planned but not implemented is explicitly labelled

[PLANNED / NOT IMPLEMENTED], and any incomplete area is labelled **[PARTIALLY IMPLEMENTED]**.

Contents

1	System Overview, Technology, and Structure	4
1.1	What Is This System?	4
1.2	Product Scope	5
1.3	Technology Stack	6
1.4	High-Level Architecture	7
1.5	Complete Project Tree	9
2	Application Routing, Authentication, and Authorization	11
2.1	Route / Blueprint Architecture	11
2.2	Request Lifecycle	13
2.3	Authentication	13
2.4	Authorization	15
2.5	Role Matrix	17
3	Database	18
3.1	Database Architecture	18
3.2	Database Table Inventory	18
3.3	Entity Relationship Diagram	20
3.4	Database Relationship Explanation	21
3.5	Constraints and Data Integrity	22
4	Business Domain	24
4.1	Employee Domain	24
4.2	User Account Domain	24
4.3	Scheduling Domain	25
4.4	Attendance Domain	26
4.5	Leave Domain	27
4.6	Overtime Domain	28
4.7	Labor Cost Domain	29
4.8	Reports	30
4.9	Audit Log	30
5	Page-by-Page Documentation	32
5.1	Login	32
5.2	Dashboard — Admin / Manager View	33
5.3	Dashboard — Employee View	34
5.4	Employees — List	34
5.5	Employee Detail	35
5.6	Employee Pay Rate	36
5.7	Departments	36
5.8	Schedule	37

5.9	Attendance	38
5.10	Leave	39
5.11	Overtime Report	39
5.12	Hours Trend Report	40
5.13	Labor Cost — Department Totals	40
5.14	Labor Cost — Employee Breakdown	41
5.15	Audit Log	41
5.16	Recent Activity	42
5.17	Error Pages	42
5.18	Admin Dashboard Deep Dive	43
6	Frontend Architecture and Security	44
6.1	Frontend Architecture	44
6.2	Design System	45
6.3	Security Architecture	46
6.4	Security Threat Model	48
7	Testing, Data Flows, and Business Rules	50
7.1	Testing Architecture	50
7.2	Important Test Cases	50
7.3	Important System Flows	51
7.4	Business Rules Catalog	53
7.5	Error Handling	56
8	Deployment, Maintenance, Limitations, and Roadmap	58
8.1	Deployment Architecture	58
8.2	Configuration	58
8.3	How to Add a New Feature	59
8.4	How to Add a New Page	60
8.5	How to Modify Existing Business Logic	60
8.6	Known Limitations	60
8.7	Recommended Roadmap	61
9	Developer Quick Reference	63
9.1	One-Page System Map	63
9.2	Important Files Cheat Sheet	63
9.3	Glossary	65
9.4	Final System Summary	66

PART 1

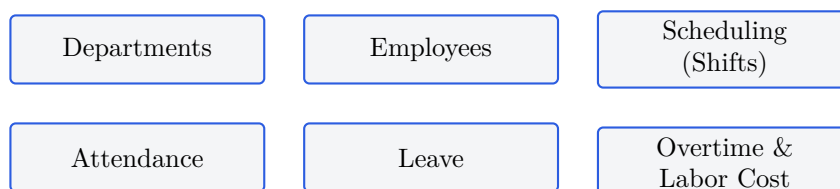
System Overview, Technology, and Structure

1.1 What Is This System?

The Workforce Management Platform is a server-rendered web application that helps a single organization answer the practical, day-to-day questions a business asks about its hourly workforce:

- Who is scheduled to work, and who actually worked?
- How many hours did each employee work, and how much of that was overtime?
- Who is on approved leave, and who has a leave request waiting?
- How much is labor costing the organization, per department?
- What privileged changes has anyone made recently?

The system is built around six core domain concepts, each with its own models and service module:



Two of these concepts are deliberately kept separate and are never conflated anywhere in the codebase:

- **Scheduling** (the `Shift` model) represents *planned* work — what an employee is expected to work.
- **Attendance** (the `AttendanceEntry` model) represents *actual* work — what really happened, recorded via clock-in/clock-out.

Working hours, overtime, and labor cost are all *derived* from these two record types rather than stored as independent numbers – see Chapter 4 for exactly how.

Who Uses It

Three roles share the same application, each seeing a different slice of it, enforced by `AccessScope` (Chapter 2):

- **Admin** — full access to one organization: every employee, every department, pay rates, labor cost detail, the audit log, and every management action.

- **Manager** — restricted to the departments they are assigned to manage (`department_managers`). Can manage schedules, attendance, and leave for their own people, and see department-level cost *totals* — never an individual employee's pay rate or exact cost.
- **Employee** — self-service only: their own schedule, their own attendance (clock in/out), their own leave requests.

1.2 Product Scope

The table below reflects what actually exists in the codebase today, verified against the route and service modules in `app/routes/` and `app/services/`.

Capability	Status	Description
Authentication	Implemented	Email/password login, Argon2 hashing, account lockout after repeated failures, absolute session expiry.
User accounts & roles	Implemented	<code>admin</code> / <code>manager</code> / <code>employee</code> , enforced by DB <code>CHECK</code> constraint and <code>AccessScope</code> .
Departments	Implemented	CRUD (<code>admin</code>), soft-deactivation, manager assignment via <code>department_managers</code> .
Employees	Implemented	CRUD, employment status lifecycle (active/inactive/terminated), department assignment.
Scheduling (shifts)	Implemented	Draft/published/cancelled shifts, employee assignment, overlap and leave-conflict prevention.
Check-in / check-out	Implemented	Self-service and admin/manager on-behalf-of clock-in/out, single-open-entry enforcement.
Breaks	[PARTIALLY IMPLEMENTED]	A single <code>break_minutes</code> integer per shift/entry only — there is no separate break start/end event, no multiple-breaks-per-shift model, and no <code>BreakSession</code> table anywhere in the schema.
Attendance correction	Implemented	Admin/manager correction with a mandatory reason; stale open entries auto-flagged <code>needs_review</code> .
Working hours	Implemented	Derived from closed attendance entries only; scheduled-vs-worked comparison per employee/day.
Overtime	Implemented	Configurable daily/weekly tiered policy, pure-function tiering, no double-counting across tiers.
Leave	Implemented	Request/approve/reject/cancel lifecycle, overlap prevention, shift-conflict detection on approval.
Leave balances / accrual	[PLANNED / NOT IMPLEMENTED]	<code>LeaveType</code> is a catalog only; no "days remaining" ledger exists.
Labor cost	Implemented	Decimal, per-line-item rounded cost; department totals (admin/manager) and per-employee breakdown (admin only).

Capability	Status	Description
Paid leave costing	[PLANNED / NOT IMPLEMENTED]	<code>LeaveType.is_paid</code> is stored but never read by the labor-cost module; approved leave currently contributes \$0 to every cost figure (documented explicitly in <code>app/services/labor_cost.py</code>).
Reports	Implemented	Overtime-hours report and hours-trend report, both department-scoped.
Audit log	Implemented	Append-only, admin-only, paginated, date-range filtered; a lighter "Recent Activity" view over the same data.
Reduced information density: workforce demand forecasting, AI-assisted scheduling, schedule optimization	[PLANNED / NOT IMPLEMENTED]	Explicitly out of scope for MVP 1 per <code>CLAUDE.md</code> .

1.3 Technology Stack

Every entry below is a real, currently-used dependency, verified against `requirements.txt` and the import statements in `app/`.

Python 3.14

What: The application's implementation language. **Why:** Type hints, dataclasses, and the standard `zoneinfo` module (used throughout for timezone-aware date arithmetic) are used extensively across the service layer. **Where:** Everywhere under `app/`.

Flask 3.1

What: A minimal WSGI web framework. **Why:** Matches the project's "modular monolith, not a microservice framework" architectural preference (`CLAUDE.md`) — Flask imposes very little structure of its own, leaving the routes/services/models layering to be an explicit project convention rather than a framework-enforced one. **Where:** `create_app()` in `app/__init__.py`; every blueprint in `app/routes/`. **Responsibility:** HTTP request/response handling, routing, session cookies, template rendering.

SQLAlchemy 2.0 (ORM) + PostgreSQL

What: SQLAlchemy is the Object-Relational Mapper; PostgreSQL is the relational database. **Why:** The domain has many real relational invariants (no overlapping shifts, no double clock-in, effective-dated pay rates and overtime policies) that are enforced with PostgreSQL-specific features — `EXCLUDE` constraints using `gist` indexes over `tstzrange/daterange`, partial unique indexes, and composite foreign keys — not just application code. **Where:** `app/models/*.py` (declarative models, using the 2.0-style `Mapped[...]` annotations); every `db.session` query in `app/services/*.py`. **Responsibility:** Schema definition, query construction, transaction management.

Flask-Migrate (Alembic)

What: Database migration tooling built on Alembic. **Why:** Schema changes must go through versioned, reviewable migration scripts rather than ad hoc DDL (`.claude/rules/database.md`).

Where: `migrations/versions/0001` through `0016`. **Responsibility:** Applying and reversing schema changes in a controlled, ordered sequence.

Flask-Login

What: Session-based authentication helper. **Why:** Provides `current_user`, the `login_required` decorator, and the user-loader hook this project uses to invalidate a session the instant an account is deactivated or locked (see Chapter 2). **Where:** `app.extensions.login_manager`; `app/models/user.py`'s `load_user`; `app/auth/decorators.py`.

Flask-WTF

What: WTForms integration with built-in CSRF protection. **Why:** Every state-changing form in the application needs CSRF protection and server-side field validation; Flask-WTF provides both with one consistent pattern. **Where:** `app.extensions.csrf`; every form class in `app/forms.py`.

argon2-cffi

What: A Python binding for the Argon2 password-hashing algorithm. **Why:** Argon2 is the current recommended password hash for new applications (memory-hard, resistant to GPU cracking), used here instead of an older algorithm like `bcrypt` or `PBKDF2`. **Where:** `app/auth/passwords.py` (`hash_password` / `verify_password`), wrapping `argon2.PasswordHasher`.

Gunicorn

What: A production-grade WSGI HTTP server for Python. **Why:** Flask's own development server is explicitly unsuitable for production; Gunicorn is the process that actually serves requests in the Docker image. **Where:** Dockerfile's CMD: `gunicorn -b 0.0.0.0:8000 wsgi:app`.

HTML, CSS, and Vanilla JavaScript

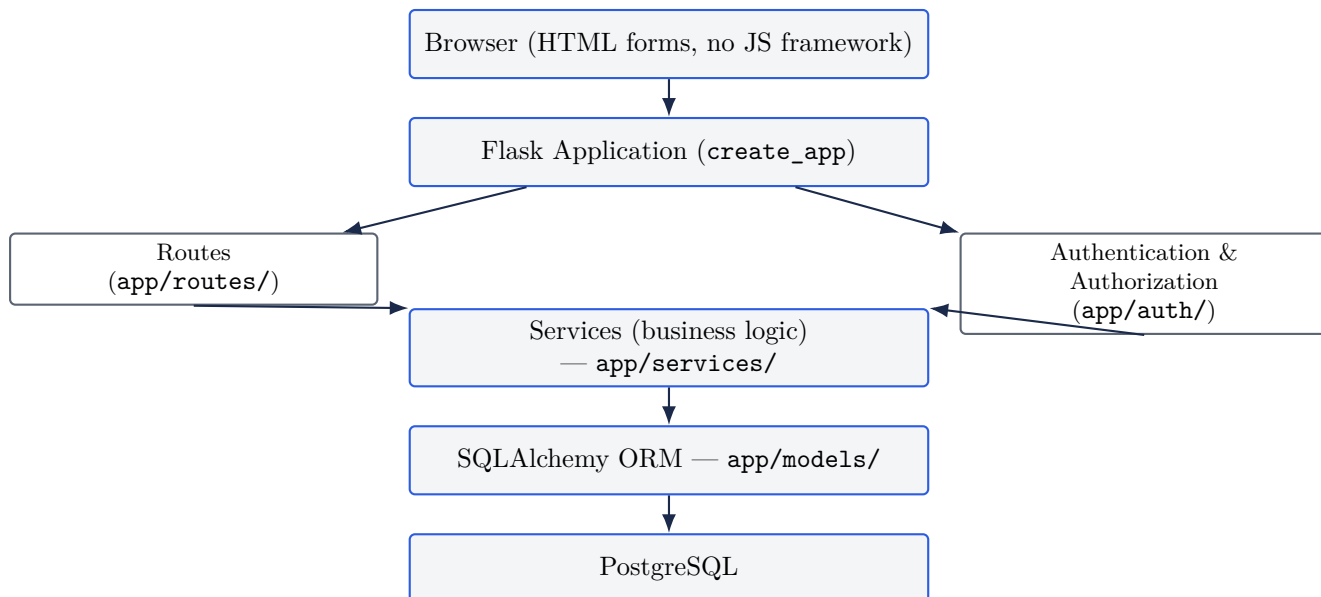
What: Server-rendered Jinja2 templates, a hand-written CSS design-token system, and a small amount of unframeworked JavaScript. **Why:** `CLAUDE.md` explicitly forbids introducing a frontend framework (React/Vue/Angular) without justification; the UI is rendered entirely server-side. **Where:** `templates/`, `static/css/`, `static/js/main.js`. See Chapter 6 for the design-system details.

pytest

What: The test framework. **Why:** Used for unit, integration, and route-level tests across the whole codebase (458 tests at the time of writing). **Where:** `tests/`. See Chapter 7.

1.4 High-Level Architecture

The application is a **modular monolith**: a single deployable Flask process, internally organized into clearly separated layers, rather than a collection of independently deployed services. This is a deliberate choice (`.claude/rules/architecture.md`) — the domain is not large enough to justify the operational cost of microservices, and a monolith keeps transactional integrity simple (a leave approval that also writes an audit entry is one database transaction, not a distributed one).



Why Routes Stay Thin

Every route module’s docstring states the same rule: a route builds an `AccessScope` from the signed-in user and delegates *all* authorization and data access to a service function — no route queries the database directly (the one historical exception, `app.routes.audit`, was fixed to go through `app.services.audit.list_entries` instead, per that module’s own “Round C fix” docstring). This keeps HTTP concerns (form parsing, flashing messages, redirects) separate from business rules, and means a business rule is only ever expressed once.

Why Business Logic Belongs in Services

Every function in `app/services/*.py` takes the caller’s `AccessScope` as its first argument and re-checks authorization itself, *independent of whatever the route layer already checked*. This “belt-and-suspenders” pattern means that even if a route were missing a decorator, the service function underneath would still refuse an unauthorized call. It also means services are usable from multiple entry points (a route, a CLI command, a future API) without re-implementing authorization each time.

Why Models Represent Persistence Only

Models in `app/models/` are declarative SQLAlchemy classes with constraints and relationships, and carry essentially no business logic of their own (a few carry a `__repr__` for debugging and nothing else). Validation and business rules live in the service layer; the database’s own constraints (Chapter 3) are the last line of defense, not the primary one.

How Authorization Is Enforced

See Chapter 2 for the full authorization model. In summary: a route decorator (`login_required` / `role_required`) provides a first, coarse check; every service function re-checks the caller’s role and organizational/department scope before touching data; and several database constraints (composite foreign keys tying a child row’s organization to its parent’s) make a cross-tenant write impossible even if every application-level check were somehow bypassed.

1.5 Complete Project Tree

The tree below is the actual top-level layout of the repository (test cache, `.git`, and Python bytecode caches omitted), matching `.claude/rules/project-structure.md`'s canonical layout.

```

job/
|-- app/
| |-- auth/ Authentication & authorization helpers
| | |-- decorators.py login_required / role_required
| | |-- passwords.py Argon2 hashing wrapper
| | |-- redirects.py Open-redirect-safe "next" handling
| | |-- scope.py AccessScope + get_scoped_or_404
| | '-- service.py Login business logic, logout policy
|-- models/ SQLAlchemy declarative models (13 tables)
|-- routes/ Flask blueprints (thin HTTP layer)
|-- services/ Business logic (the bulk of the application)
|-- cli.py Flask CLI commands (attendance flag-stale, seed)
|-- config.py Environment-driven configuration classes
|-- errors.py 400/403/404/413/500 error handlers
|-- extensions.py Flask extension singletons (db, login, csrf...)
|-- forms.py WTForms form classes
| '-- __init__.py create_app() application factory
|-- migrations/
| '-- versions/ 16 ordered Alembic migration scripts
|-- templates/ Jinja2 templates (one subfolder per blueprint)
| |-- components/ Shared macros: icons, badges, sidebar, topbar
| '-- errors/ 400/403/404/413/500 error pages
|-- static/
| |-- css/ tokens.css, base.css, components.css, layout.css
| |-- fonts/ Self-hosted variable fonts (woff2)
| '-- js/main.js Small vanilla-JS behavior layer
|-- tests/ pytest suite (458 tests)
| |-- unit/ Pure-function tests, no DB/Flask app
| |-- integration/ Service-layer tests against a real test DB
| '-- routes/ Full HTTP request/response tests
|-- docs/ This documentation (see docs/system-documentation/)
|-- CLAUDE.md Project-wide AI/developer instructions
|-- .claude/rules/ Per-topic project rules (architecture, security, ...)
|-- docker-compose.yml app + postgres services for local/containerized use
|-- Dockerfile Production image: Gunicorn serving wsgi:app
|-- requirements.txt Pinned Python dependencies
'-- wsgi.py WSGI entrypoint (create_app())

```

Important Directories

app/routes/ — One blueprint per domain area (attendance, audit, auth, dashboard, departments, employees, labor_cost, leave, main, schedule). Each file's own docstring restates the "thin route, delegate to services" rule.

app/services/ — The actual business logic: attendance, audit, availability, departments, employees, labor_cost, leave, overtime, pay_rates, reports, scheduling, working_hours. This is the directory a new developer should read first to understand what the system actually does (see Chapter 9's "How to add a feature").

app/auth/ — Everything authorization-related lives here, isolated from both routes and services so it can be reasoned about (and reviewed) as a single unit. `AccessScope` in `scope.py` is the single data structure every authorization decision in the codebase is built from.

migrations/versions/ — The authoritative history of the schema. Because Alembic migrations are additive scripts, the actual deployed schema is the cumulative effect of all 16 files, not just the current state of `app/models/` (see Chapter 3, §3.4, on why the two are

cross-checked rather than assumed identical).

tests/ — Organized by test *kind*, not by feature: **unit/** needs no database or Flask app context at all (e.g. the overtime tiering math); **integration/** exercises a service function against a real PostgreSQL test database; **routes/** drives the application exactly as an HTTP client would, including session cookies and CSRF tokens.

PART 2

Application Routing, Authentication, and Authorization

2.1 Route / Blueprint Architecture

The application registers ten Flask blueprints in `create_app()` (`app/__init__.py`), plus one inline route (`/healthz`) registered directly on the app for container health checks. The table below is a complete map of every route in the codebase, extracted directly from the blueprint files.

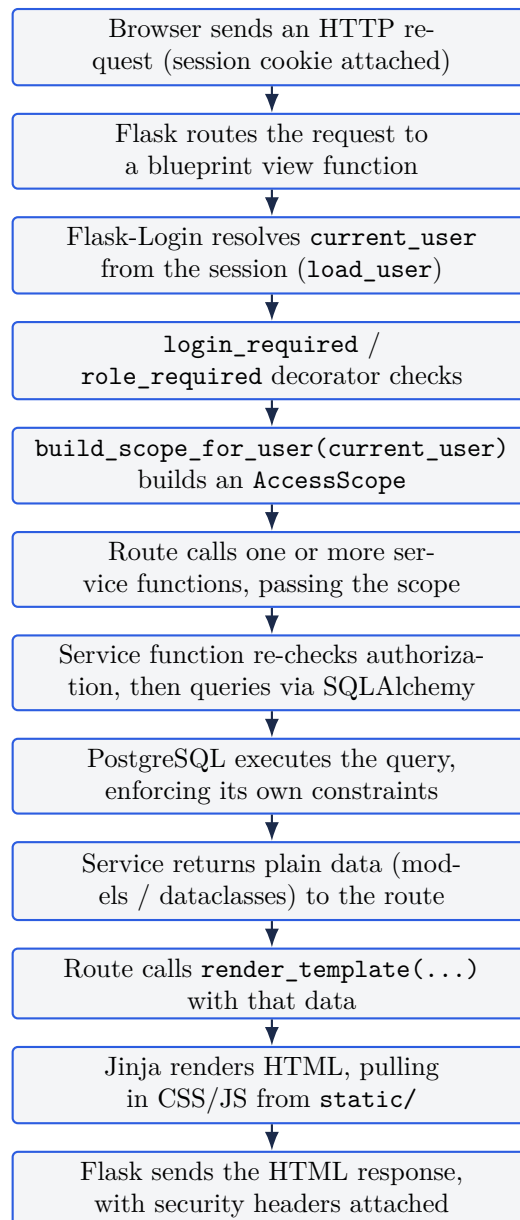
Blueprint	Route	Method	Role	Key service call(s)	Template
main	/	GET	any	— (redirect)	redirects to dashboard.index
auth	/login	GET/POST	public	<code>auth.service.authenticate</code>	<code>auth/login.html</code>
auth	/logout	POST	any	<code>flask_login.logout_user</code>	redirect
dashboard	/dashboard	GET	any	<code>reports.*</code> (many)	<code>dashboard/admin.html</code> or <code>dashboard/employee.html</code>
dashboard	/reports/overtime	GET	admin, manager	<code>reports.overtime_summary</code>	<code>reports/overtime.html</code>
dashboard	/reports/hours-trend	GET	admin, manager	<code>reports.hours_trend</code>	<code>reports/hours_trend.html</code>
attendance	/attendance	GET	any	<code>reports.attendance_entries_with_context</code> , <code>reports.current_attendance_status</code>	<code>attendance/list.html</code>
attendance	/attendance/clock-in	POST	any	<code>attendance.clock_in</code>	redirect
attendance	/attendance/<id>/clock-out	POST	any	<code>attendance.clock_out</code>	redirect
attendance	/attendance/<id>/correct	POST	admin, manager	<code>attendance.correct_entry</code>	redirect
schedule	/schedule	GET	any	<code>scheduling.list_shifts</code>	<code>schedule/list.html</code>
schedule	/schedule	POST	admin, manager	<code>scheduling.create_shift</code>	redirect / re-render
schedule	/schedule/<id>	POST	admin, manager	<code>scheduling.update_shift</code>	redirect
schedule	/schedule/<id>/assign	POST	admin, manager	<code>scheduling.assign_employee</code>	redirect

Blueprint	Route	Method	Role	Key service call(s)	Template
schedule	/schedule/ <id>/publish	POST	admin, manager	scheduling.publish_shift	redirect
schedule	/schedule/ <id>/cancel	POST	admin, manager	scheduling.cancel_shift	redirect
leave	/leave	GET	any	leave.list_leave_requests	leave/list.html
leave	/leave	POST	any	leave.request_leave	redirect
leave	/leave/ <id>/approve	POST	admin, manager	leave.approve_leave	redirect
leave	/leave/<id>/reject	POST	admin, manager	leave.reject_leave	redirect
leave	/leave/<id>/cancel	POST	any	leave.cancel_leave	redirect
employees	/employees	GET	admin, manager	employees.list_employees	employees/list.html
employees	/employees	POST	admin	employees.create_employee	redirect / re-render
employees	/employees/<id>	GET	any	employees.get_employee, employees.get_linked_user	employees/detail.html
employees	/employees/<id>	POST	admin, manager	employees.update_employee	redirect
employees	/employees/ <id>/terminate	POST	admin	employees.terminate_employee	redirect
employees	/employees/ <id>/pay-rate	GET	admin	pay_rates.list_pay_rate_history	employees/pay_rate.html
employees	/employees/ <id>/pay-rate	POST	admin	pay_rates.set_pay_rate	redirect
departments	/departments	GET	admin, manager	departments.list_departments	departments/list.html
departments	/departments	POST	admin	departments.create_department	redirect / re-render
departments	/departments/<id>	POST	admin	departments.update_department	redirect
departments	/departments/ <id>/deactivate	POST	admin	departments.deactivate_department	redirect
labor_cost	/labor-cost	GET	admin, manager	labor_cost.department_cost_summary	labor_cost/index.html
labor_cost	/labor-cost/ employees/<id>	GET	admin	labor_cost.range_cost_for_employee	labor_cost/employee_detail.html
audit	/audit-log	GET	admin	audit.list_entries	audit/list.html
audit	/recent-activity	GET	admin	reports.recent_activity	audit/recent_activity.html
(app-level)	/healthz	GET	public	—	JSON {"status": "ok"} }

"Role" above is the minimum role_required(...) on the route; "any" means login_required only (every authenticated role may reach the route, though what it returns still differs by role — e.g. /attendance shows only the caller's own entries for an employee).

2.2 Request Lifecycle

Every page load in the application follows the same path. This is the concrete version of the generic layered architecture from Chapter 1, now including where `AccessScope` is built and where the Jinja/CSS layer fits in.



The security headers in the last step (`X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, a strict `Content-Security-Policy`, and conditionally `Strict-Transport-Security`) are attached by an `@app.after_request` hook in `app/__init__.py` to *every* response, not just authenticated pages. See Chapter 6, §6.1, for why the CSP can afford to be this strict (no external scripts, no inline styles, no CDN fonts).

2.3 Authentication

Login

`POST /login` (`app/routes/auth.py`) delegates credential checking entirely to `app.auth.service.authenticate`. That function:

1. Looks up the user by email. If none exists, the account is inactive, or the account is currently locked, it still runs a *real* Argon2 verification against a fixed dummy hash (`_burn_dummy_verification_time`) before returning failure — this keeps response timing statistically indistinguishable across every failure reason, closing a user-enumeration timing side-channel.
2. On a genuine password mismatch, increments `failed_login_count` and, once it reaches `MAX_FAILED_LOGIN_ATTEMPTS` (5), sets `locked_until` to 15 minutes in the future (`LOCKOUT_DURATION`).
3. On success, resets the failure counter, records `last_login_at`, and stages an audit log entry.
4. Every branch returns the identical, generic "Invalid email or password." message — the UI never reveals *which* of "no such account" / "wrong password" / "account locked" applies.

On success, the route (`app/routes/auth.py`) calls `session.clear()` *before* `login_user(...)` — this discards any pre-authentication session state (including the already-validated CSRF token) so nothing from before login carries into the authenticated session, a defense against session fixation. It then sets `session.permanent = True`, so the cookie expires `PERMANENT_SESSION_LIFETIME` (12 hours) after login regardless of activity, and redirects to a same-site-validated `next` target (`app.auth.redirects.get_safe_redirect_target` — see §6.1 for the open-redirect defense).

Password Hashing

Passwords are hashed with Argon2 (`argon2.PasswordHasher`'s defaults) in `app/auth/passwords.py`. The plaintext password is never stored, logged, or included in an audit entry anywhere in the codebase (verified: `audit_service.record`'s `changes` dict is built explicitly, field by field, at every call site — there is no code path that could pass a whole form or a raw password through).

Sessions

Flask-Login's signed session cookie is the only session mechanism — there is no separate server-side session table. This means the `load_user` hook in `app/models/user.py` (invoked on *every* request for an authenticated user) is the actual enforcement point for "has this account been deactivated or locked out since the cookie was issued?" — it returns `None` (forcing re-authentication) if `is_active` is false or `locked_until` is still in the future, even for an otherwise validly-signed cookie.

Account Lockout

Implemented exactly as described under Login above: `failed_login_count` (a small integer, `CHECK ... >= 0`) and `locked_until` (nullable `timestampz`) on the `users` table. A wrong-password attempt against an already-*expired* lock resets the counter to 1 (treating it as the first attempt of a fresh window) rather than incrementing a stale count back over the threshold — see the inline comment in `authenticate` for why the naive version of this logic could re-lock an account indefinitely from one further wrong guess per lockout period.

Unauthorized Requests

An unauthenticated request to any `login_required` route is redirected to `/login` by Flask-Login (`login_manager.login_view = "auth.login"`), with a flashed message ("Please log in to access this page."). A request from an authenticated user lacking the required role receives a plain **403** (`abort(403)` inside `role_required`), rendered by `templates/errors/403.html`.

2.4 Authorization

This is the most important subsystem in the codebase to understand correctly, because a mistake here is a data-leak or privilege-escalation bug, not a cosmetic one.

AccessScope

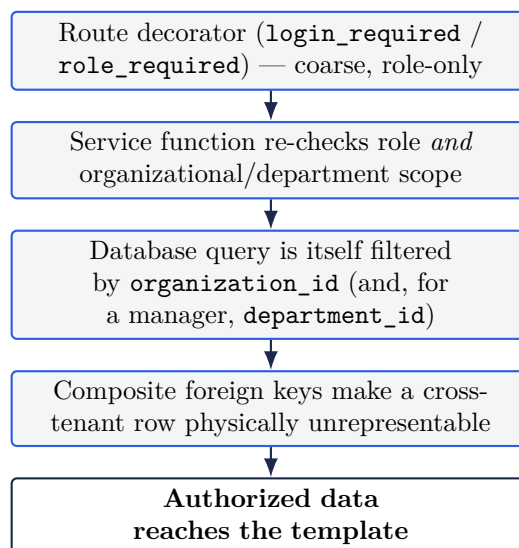
Every authorization decision in the application is built from one frozen dataclass, `app.auth.scope.AccessScope`.

```
@dataclass(frozen=True)
class AccessScope:
    user_id: int
    organization_id: int
    role: str # 'admin' / 'manager' / 'employee'
    department_ids: frozenset[int] # populated only for 'manager'
    employee_id: int | None # populated for 'employee' (and any
                           # admin/manager who also has a
                           # linked employee record)
```

`build_scope_for_user(current_user)` constructs this once per request, at the top of every route, by querying `department_managers` for a manager's assigned department ids. Every route in the table in §2.1 calls this before doing anything else.

Defense in Depth

Authorization is checked at up to three independent layers, and the codebase's own module docstrings repeatedly call this out as deliberate:



Why not rely on hiding UI elements? A hidden button is a client-side courtesy, not a security control — a browser's developer tools or a hand-crafted `curl` request can submit any form regardless of what the rendered HTML shows. Every service function in this codebase enforces its own rule regardless of what the calling route or template does, so an attacker who reaches a route directly (bypassing the UI entirely) gets exactly the same rejection a normal user browsing the hidden feature would have. `tests/routes/test_idor_sweep.py` exists specifically to exercise this: it drives routes directly with an out-of-scope id and asserts a 404, not just checks that a link is absent from a page.

The IDOR Defense: `get_scoped_or_404`

`app.auth.scope.get_scoped_or_404(model, obj_id, scope)` is the standard way to fetch a single tenant-owned row. It always filters by `organization_id`, and additionally filters by `department_id` when the model has that column and the caller is a manager. Critically, an out-of-scope row returns **404**, not **403** — so a caller cannot distinguish "this id doesn't exist" from "this id exists but belongs to someone else," which would otherwise leak the existence of other organizations'/departments' data through the response code alone. Several models without a `department_id` column of their own (`AttendanceEntry`, `LeaveRequest`) implement the same 404-not-403 pattern by hand, via a join to the owning employee (e.g. `attendance._get_entry_for_scope`).

Organization Scoping

Every tenant-owned table carries an `organization_id` column, and every service query filters on it. This is reinforced at the database level: every child table's foreign key to a parent (e.g. an employee's department, a shift's employee) is a *composite* foreign key on (`parent_id`, `organization_id`), referencing a matching composite unique constraint on the parent — so it is not merely application-code discipline that prevents, say, an employee row from pointing at a department in a different organization; the schema itself makes it impossible.

Manager Department Scoping

A manager's `department_ids` (Chapter 2's `AccessScope`) comes from the `department_managers` join table. Every manager-reachable service function filters its query by `department_id` `IN scope.department_ids` (directly, or via a join to the owning employee for tables with no department column). Because this filter is applied *before* any caller-supplied filter parameter (e.g. `?employee_id=` or `?department_id=` on several list views), a manager passing an out-of-scope id gets an *empty intersection*, never that other department's data — this composition order is exercised directly by tests such as `test_manager_filtering_by_an_unmanaged_employee_sees_no_data`.

Admin Privileges

An admin has no `department_ids` restriction at all — every manager-or-admin-reachable service function's manager-scoping branch is skipped entirely for `role == "admin"`. Admin is still confined to their own `organization_id` like every other role (there is no concept of a cross-organization "super-admin" anywhere in the schema or code).

Manager Privileges

Everything an admin can do within their managed departments, with two explicit, hard-coded exceptions the codebase calls "rule A4": a manager may see a department's aggregate labor cost *total* (`labor_cost.department_cost_summary`) and an employee's overtime *hours* (`reports.overtime_summary`, which reads only `LineItem.hours/.category`, never `.rate` or `.cost`) — but never an individual employee's hourly pay rate or per-employee cost breakdown. Those two views (`/employees/<id>/pay-rate` and `/labor-cost/employees/<id>`) are hard-coded `role_required("admin")`, not `role_required("admin", "manager")` like most of the rest of the application.

Employee Privileges

Self-service only: their own schedule (published shifts only), their own attendance (clock in/out, view history), their own leave requests. An employee-role `AccessScope` has no `department_ids`

at all (an empty frozenset) — every service function’s employee branch checks identity (`scope.employee_id == target_id`) rather than department membership.

2.5 Role Matrix

Verified directly against the `role_required(...)` decorators and each service function’s own internal role check — not guessed from route names.

Feature	ADMIN	MANAGER	EMPLOYEE
View own dashboard	Org-wide overview	Managed-department overview	Personal dashboard
View / create / edit departments	Full	View only	No access
Deactivate a department	Yes	No	No
View employee list	Org-wide	Managed departments	No access (route is admin/manager only)
View one employee’s detail page	Any employee	Managed-department employees	Own record only
Create / edit an employee	Yes	Edit only, managed departments	No
Terminate an employee	Yes	No	No
View / set an employee’s pay rate	Yes	No	No
View a department’s labor cost total	Yes	Managed departments	No
View an employee’s labor cost breakdown	Yes	No	No
View / create shifts	Org-wide	Managed departments	View own published shifts only
Assign / publish / cancel a shift	Yes	Managed departments	No
Clock in / out (self)	Yes	Yes	Yes
Clock in/out on another’s behalf	Yes	Managed departments	No
Correct an attendance entry	Yes	Managed departments	No (not even their own)
Request leave (self)	Yes	Yes	Yes
Request leave on another’s behalf	Yes	Managed departments	No
Approve / reject leave	Yes	Managed departments	No
Cancel a leave request	Any	Managed departments	Own, while still pending
View overtime / hours-trend reports	Yes	Managed departments	No
View the audit log / recent activity	Yes	No	No

PART 3

Database

3.1 Database Architecture

PostgreSQL is the single source of truth for all persistent data. The application never treats an in-memory or derived value as authoritative when a real database record could answer the same question — for example, worked hours are always recomputed from closed `attendance_entries` rows, never cached on the `Employee` row itself.

SQLAlchemy 2.0's declarative style (`Mapped[...]` type annotations) defines every table in `app/models/*.py`. Each model file's own docstring documents *why* its constraints exist, not just what they are — this documentation quotes many of those docstrings directly, since they are the authoritative rationale.

Model → Migration → Table

Alembic migrations in `migrations/versions/` are the actual, applied schema history; `app/models/*.py` is SQLAlchemy's in-Python *description* of that schema, used for query building and (via `compare_type=True` in `Migrate.init_app`) for autogenerating future migrations. The two are cross-checked here rather than assumed identical, per this project's own documentation requirement.

Several important constraints in the schema are **hand-written**, not autogenerated, because SQLAlchemy's Alembic autogeneration cannot produce PostgreSQL's `EXCLUDE ... USING gist` DDL on its own — this is called out explicitly in the docstrings of `Shift.ex_shifts_employee_no_overlap`, `AttendanceEntry.ex_attendance_entries_employee_no_overlap`, `LeaveRequest.ex_leave_requests_employee_no_overlap`, `EmployeePayRate.ex_employee_pay_rates_employee_no_overlap`, and `OvertimePolicy.ex_overtime_policies_organization_no_overlap`. No discrepancy between the model definitions and the migration scripts was found during the preparation of this document; both were read directly for every table below.

Naming Convention

`app/models/base.py` registers a SQLAlchemy naming convention before any table is defined, so every constraint/index Alembic autogenerates gets a predictable, greppable name (`ix_`, `uq_`, `ck_`, `fk_`, `pk_` prefixes). This is why constraint names appear consistently throughout this chapter and in the service-layer error handling that matches specific constraint names (e.g. `uq_attendance_entries_employee_id_open`) to translate a raw database error into a friendly `ValidationError`.

3.2 Database Table Inventory

Thirteen tables exist in the schema, verified against `migrations/versions/0001` through `0016` and every model file.

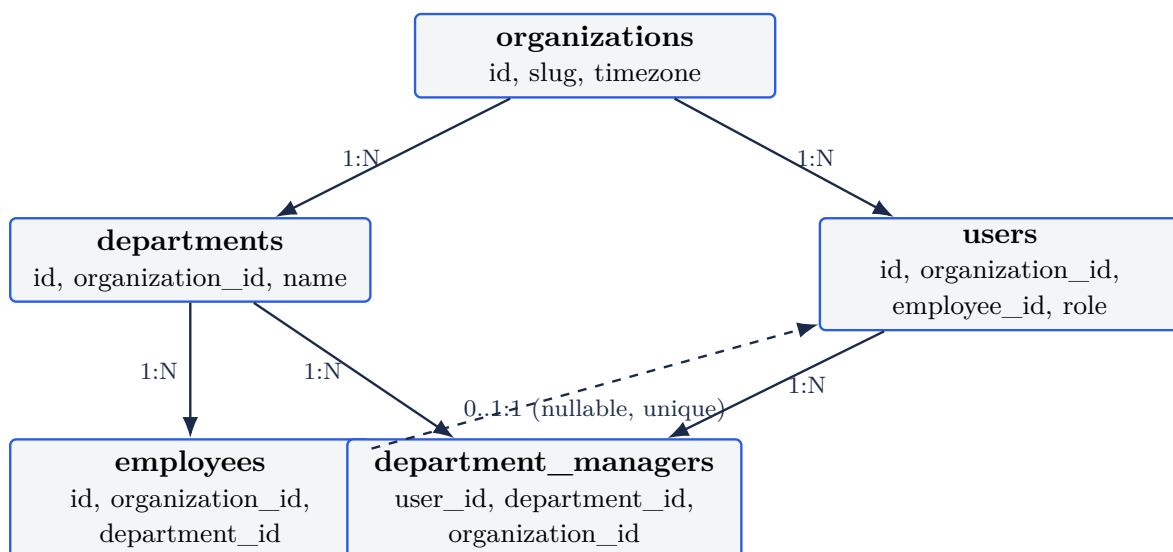
Table	Purpose	PK	Important columns
organizations	The tenant boundary; every other table is scoped to one.	id	slug (unique), timezone, currency_code (CHAR 3), is_active
departments	Organizational units employees belong to.	id	organization_id, name (case-insensitive unique per org), code, is_active
employees	The HR record: a person who can be scheduled and tracked, distinct from their login.	id	organization_id, department_id, employee_number, first_name, last_name, email (nullable, unique when present), employment_status, pay_type ('hourly' only), hired_on, terminated_on
users	A login/authentication account, optionally linked to an employee.	id	organization_id, employee_id (nullable, unique), email (unique, case-insensitive), password_hash, role, is_active, failed_login_count, locked_until
department_managers	Join table: which manager-role users may act on which departments.	composite (user_id, department_id)	organization_id
shifts	A single planned work period (<i>scheduling</i> , not attendance).	id	organization_id, department_id, employee_id (nullable = unassigned), starts_at, ends_at, business_date, break_minutes, status, published_at
attendance_entries	A single actual work period, recorded via clock-in/clock-out.	id	organization_id, employee_id, shift_id (nullable), started_at, ended_at (nullable while open), business_date, break_minutes, status, source, edit_reason
overtime_policies	An organization's effective-dated overtime configuration.	id	organization_id, daily_threshold_hours, weekly_threshold_hours, week_start_day, effective_from/_to
overtime_tiers	One multiplier tier belonging to a policy.	id	policy_id, scope (daily/weekly), tier_order, from_hours, to_hours, multiplier

Table	Purpose	PK	Important columns
leave_types	A small per-organization catalog of leave categories.	id	organization_id, code, name, is_paid, requires_approval, blocks_scheduling, is_active
leave_requests	An employee's request for time off and its approval lifecycle.	id	organization_id, employee_id, leave_type_id, starts_at/ends_at, status, decided_by_user_id, decided_at, decision_note
employee_pay_rates	An employee's effective-dated hourly rate history.	id	employee_id, organization_id, hourly_rate (NUMERIC(10,4)), effective_from/_to
audit_logs	Append-only record of privileged/sensitive actions.	id	organization_id (nullable), actor_user_id (nullable), action, entity_type, entity_id, changes (JSONB), ip_address (INET), created_at

3.3 Entity Relationship Diagram

The schema is shown in two parts for readability: the organizational identity hierarchy, and the workforce/attendance domain tables that hang off it. Every arrow below corresponds to a real foreign key (several of them composite, tying a child row's `organization_id` to its parent's) verified against the model files in §3.2.

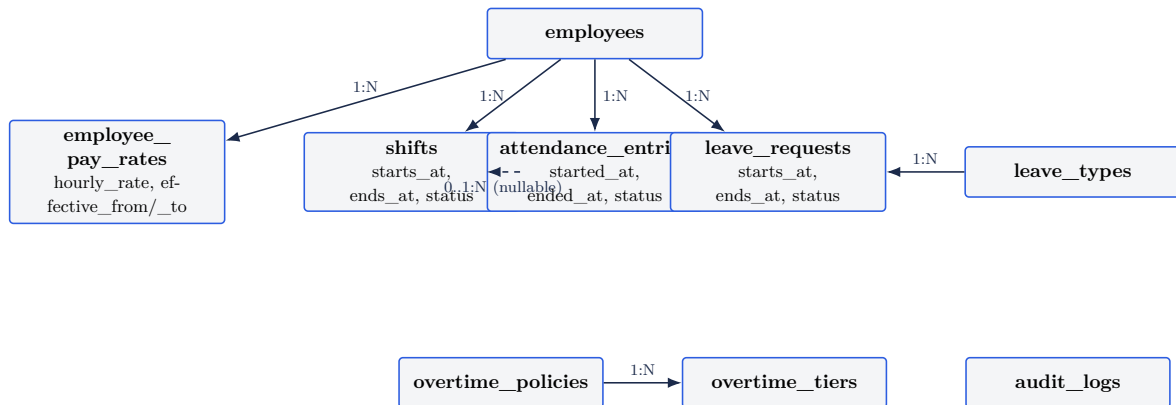
Organizational Identity



The dashed **employees**→**users** edge is deliberately 0..1:1 and *nullable*: an admin or manager account need not have a linked HR record at all, but a `role = 'employee'` account *must*

(enforced by the `employee_role_requires_employee_id` CHECK constraint, not just application code).

Workforce and Attendance Domain



`overtime_policies` and `audit_logs` both belong to `organizations` directly (not shown as an edge above to avoid crossing every other line); `audit_logs.actor_user_id` is a nullable foreign key to `users` (nullable because a failed login against a nonexistent email establishes no user at all). The dashed `shifts`→`attendance_entries` edge is the "matched shift" link set at clock-in time by `attendance._match_shift` — an attendance entry can exist with no matched shift (`shift_id` IS NULL), representing unscheduled work or a clock-in that didn't unambiguously match exactly one published shift.

3.4 Database Relationship Explanation

```

Organization
|
|--- Department
| |
| |--- Employee
| |
| |--- User (0..1, the login account)
| |--- Shift (planned work)
| |--- AttendanceEntry (actual work)
| |--- LeaveRequest
| |--- EmployeePayRate
|
|--- DepartmentManager (User <-> Department, many-to-many)
|--- OvertimePolicy -- OvertimeTier
|--- LeaveType
|--- AuditLog
  
```

Organization → Department → Employee. A strict hierarchy: an employee cannot exist without a department, and a department cannot exist without an organization. This mirrors the real-world structure the application models, and every composite foreign key in the schema (e.g. `employees.(department_id, organization_id) → departments.(id, organization_id)`) exists specifically to prevent an employee from ever being wired to a department in a *different* organization than its own.

Employee ↔ User. Deliberately *not* the same row. An `Employee` is an HR record; a `User` is a login credential. This separation is why an admin or manager can exist with no employee

record at all (they manage the system but are not necessarily "staff" being scheduled), while every `employee-role` user must have exactly one linked employee record.

Department ↔ **User**, via **DepartmentManager**. A many-to-many join: one manager can manage several departments, and (implicitly, though not currently exercised by any route) a department could have more than one manager. A manager's entire `AccessScope.department_ids` is sourced from this one table.

Employee → **Shift**, **AttendanceEntry**, **LeaveRequest**, **EmployeePayRate**. Four independent one-to-many relationships, each representing a different kind of historical record about the same person. They are kept as separate tables specifically so that, for example, a pay rate change never has to touch (or invalidate) existing attendance rows.

OvertimePolicy → **OvertimeTier**. The only `ON DELETE CASCADE` relationship in the entire schema (every other foreign key is `RESTRICT`) — a tier has no independent meaning without its parent policy, so cascading its deletion is safe; `database.md`'s "avoid accidental cascade deletes" rule is treated as a default to override only with this kind of genuine ownership relationship, not a blanket prohibition.

3.5 Constraints and Data Integrity

The schema treats workforce data as financial-grade: money and attendance figures flow directly into payroll-adjacent calculations, so integrity is enforced at the database layer wherever practical, not only in application code.

Overlap Prevention (GiST Exclusion Constraints)

Five `EXCLUDE ... USING gist` constraints are the schema's signature integrity mechanism, each preventing a specific kind of double-booking that a simple `UNIQUE` constraint cannot express:

- `ex_shifts_employee_no_overlap` — no employee may have two overlapping, non-cancelled, assigned shifts.
- `ex_attendance_entries_employee_no_overlap` — no employee may have two overlapping attendance entries (an open entry's unbounded range still counts as overlapping anything after it starts).
- `ex_leave_requests_employee_no_overlap` — no employee may have two overlapping pending/approved leave requests.
- `ex_employee_pay_rates_employee_no_overlap` — no employee may have two overlapping effective-dated pay rate periods.
- `ex_overtime_policies_organization_no_overlap` — an organization may have at most one overtime policy in force on any given calendar day.

Each of these is paired with a *best-effort* application-level pre-check (e.g. `scheduling._check_overlap`) that gives a friendly error before ever reaching the database — but the database constraint is the actual authority under concurrency; every service function that can trigger one of these catches the resulting `IntegrityError`, inspects `error.orig.diag.constraint_name`, and re-raises a clean `ValidationError` instead of letting a raw 500 reach the user.

Partial Unique Indexes

`uq_attendance_entries_employee_id_open` (unique on `employee_id` WHERE `ended_at` IS NULL) is the actual mechanism preventing a duplicate clock-in: an employee may have at most one *unresolved* (open or needs-review) attendance entry at a time, regardless of any application-level check. Similarly, `uq_employees_organization_id_email` is unique only WHERE `email` IS NOT NULL, so multiple employees with no email on file never collide with each other.

Key CHECK Constraints

- `shifts.duration_max_24_hours` / `attendance_entries.duration_max_24_hours` — no single shift or attendance entry may span more than 24 hours, preventing an arbitrarily long entry from distorting overtime or labor cost.
- `attendance_entries.status_open_matches_ended_at_null` — `ended_at` is NULL if and only if status is open or needs_review; only closed carries an end time.
- `attendance_entries.edit_requires_edited_at_and_reason` — a correction must always carry who, when, and why together, never just one or two of the three.
- `leave_requests.decision_matches_status` (split into two CHECKs) — carefully designed so that cancelling a *previously approved* request preserves who approved it and when, distinguishing it from a request cancelled while still pending. See the extended rationale in `app/models/leave_request.py`.
- `employees.terminated_status_matches_terminated_on` — an employee's status is 'terminated' if and only if `terminated_on` is set.
- `users.employee_role_requires_employee_id` — a `role = 'employee'` login must be linked to an employee record.

Indexes

Indexes are added only where a real, identified query pattern needs them (`.claude/rules/database.md`: "add indexes only when justified"), and several were added specifically in response to slow query shapes discovered during development — for example, `ix_attendance_entries_employee_id_business_date` exists because `working_hours.worked_seconds_for_day/_week` are called in tight per-day loops by cost and report calculations, and neither pre-existing index matched that predicate shape.

Cascade Behavior

Every foreign key in the schema is ON DELETE RESTRICT **except** `overtime_tiers.policy_id`, which is ON DELETE CASCADE (a tier cannot outlive its policy) and `department_managers`'s foreign key to `users`, which is also ON DELETE CASCADE (a manager assignment has no meaning once the user account itself is gone). RESTRICT everywhere else means the schema will refuse to delete, for example, a department that still has employees — deactivation (`is_active = false`), not deletion, is how the application retires a department or an employee record, preserving historical schedules/attendance/cost data.

PART 4

Business Domain

4.1 Employee Domain

An **Employee** row is the HR record: name, employee number (organization-unique), department, employment status (**active/inactive/terminated**), optional email/phone, hire date, and (for a terminated employee) a termination date.

Creation (`employees.create_employee`, admin only) validates the target department belongs to the caller’s organization and creates the row with `employment_status` as given by the caller (the create form only offers **active/inactive** — see `app/forms.py`’s comment on why **terminated** is deliberately excluded from that choice list).

Editing (`employees.update_employee`, admin or a manager restricted to their own managed departments) accepts a fixed set of fields (name, department, contact info, employment status, weekly contract hours) — reassigning department is a plain field edit here, not a separate action.

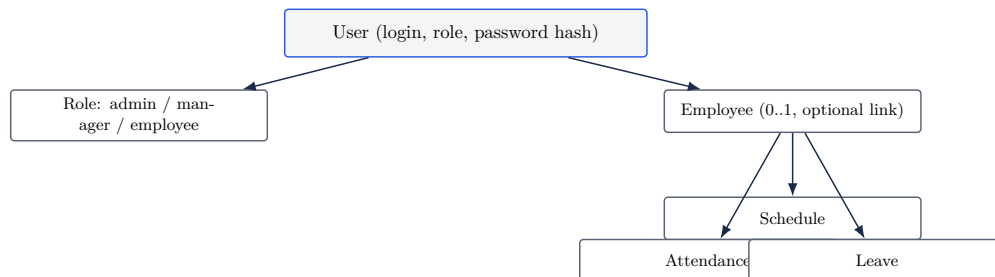
Termination (`employees.terminate_employee`, admin only) is its own dedicated action, not a generic status edit, because it must set `terminated_on` in the same step to satisfy the `terminated_status_matches_terminated_on` CHECK constraint. Terminating an employee never touches their existing shifts or attendance history — see §4.3 for why a shift already assigned to a since-terminated employee is left alone.

Employee/department relationship is mandatory (an employee cannot exist without a department) and validated on every write, not just at creation.

Employee/user relationship is optional and one-to-one (§3.4) — see §4.2.

4.2 User Account Domain

A **User** is the login identity: email, Argon2 password hash, **role**, and an optional link (`employee_id`) to an **Employee** record.



Account creation. There is no self-service sign-up route anywhere in the codebase — every **User** row is created directly (there is no `create_user` service function or route at all; the only path visible in the code is the `flask seed overtime-policy` CLI command’s neighbor

pattern, and direct database/CLI provisioning). This is a genuine gap worth calling out: **[NEEDS VERIFICATION]** how user accounts are provisioned in a real deployment is not established by the application code itself.

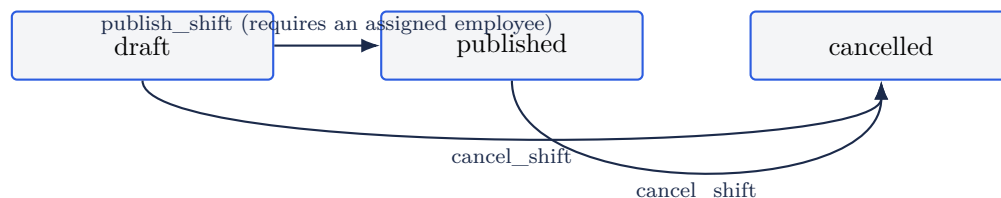
Authentication, role assignment, and what an account can access are covered in full in Chapter 2.

Account status. `is_active` (boolean) and `locked_until` (temporary, from the lock-out policy) are the two account-status mechanisms that exist. There is no separate "disabled/pending/invited" status enum — an account is either active or not.

4.3 Scheduling Domain

A **Shift** represents *planned* work: a department, an optional assigned employee, a start/end time, a break duration, and a lifecycle status.

Lifecycle



A shift is created as **draft**. Only a draft shift can be edited directly via `update_shift` (department, times, break, notes) — once published, its fields are locked except through the dedicated `assign_employee` (reassignment is allowed even after publishing, to cover a call-out), `publish_shift`, and `cancel_shift` actions. `publish_shift` requires an employee already assigned; `cancel_shift` never deletes the row (history is preserved for reporting) and works from either **draft** or **published**.

Overlap Prevention and Employee Assignment

Assigning an employee to a shift (`create_shift` with an `employee_id`, or `assign_employee`) runs three checks:

1. The employee exists in the caller's organization, is in a department the caller may act on, and is currently active.
2. No overlapping, non-cancelled shift already exists for that employee (`availability.shifts_overlapping`, backed by the database's `ex_shifts_employee_no_overlap`).
3. No approved leave (of a leave type with `blocks_scheduling = true`) overlaps the shift's time range (`availability.approved_leave_overlapping`) — this is a purely application-level check with no matching database constraint, since leave and shifts are different tables.

Department Relationships and Coverage

`scheduling.coverage_summary` answers "do we have enough people scheduled" with a deliberately minimal pair of counts (published shifts that day vs. active employees in the department) — not a full staffing model.

Effective Dates and Validation

Shifts are not effective-dated in the way pay rates/overtime policies are; `business_date` is simply the local calendar date of `starts_at` in the organization's timezone (rule A1), computed once at creation/update time by the service layer, never re-derived by the database. A shift is capped at 24 hours (`duration_max_24_hours`), and its break cannot consume the entire shift (`break_minutes_less_than_duration`).

4.4 Attendance Domain

`AttendanceEntry` represents *actual* work: who, when they started, when (if at all) they stopped, and how the record got there.

Check-In

`attendance.clock_in(scope, employee_id=None, at=None):`

- Defaults to the caller's own employee record; clocking in someone else requires admin/manager (a manager only within their own departments).
- `at` defaults to "now"; only admin/manager may set a different (backdated, never future) time — an employee can never backdate their own clock-in. A backdated time is bounded to at most 90 days in the past (`_MAX_BACKDATE`).
- Attempts to match the new entry to a single unambiguous published shift within a 60-minute grace window on either side (`_match_shift`); zero or more than one candidate leaves `shift_id` NULL rather than guessing.
- The database's partial unique index (`uq_attendance_entries_employee_id_open`) is the actual guarantee against a duplicate clock-in.

Check-Out

`attendance.clock_out(scope, attendance_entry_id, at=None)` closes an open entry, setting `ended_at` and `status = 'closed'`. It explicitly rejects an already-closed entry, and — importantly — rejects a `needs_review` entry outright: only `correct_entry` (admin/manager, with a mandatory reason) can resolve one, "since that defeats the purpose of the review flag."

Needs-Review Behavior

`attendance.flag_stale_open_entries(cutoff_hours=16)` (run via the flask `attendance flag-stale` CLI command, intended to be scheduled externally — there is no in-process scheduler) marks every open entry older than the cutoff as `needs_review`, *without ever inventing an end time*. This is the entirety of the "richer attendance state machine": there is no automatic break tracking, no multi-step state machine beyond `open/needs_review/closed`. Any richer workflow (explicit break start/end, multiple open sessions) described elsewhere as an aspiration is **[PLANNED / NOT IMPLEMENTED]**, not implemented — see §1.2's product scope table.

Correction

`attendance.correct_entry` (admin/manager only, mandatory `edit_reason`) is the *only* way an entry's times may change after creation. It can set `started_at`, `ended_at`, and `break_minutes`; setting `ended_at` always closes the entry (satisfying the CHECK that ties status to `ended_at` nullability). It is the only mechanism that can resolve a `needs_review` entry.

Break Handling

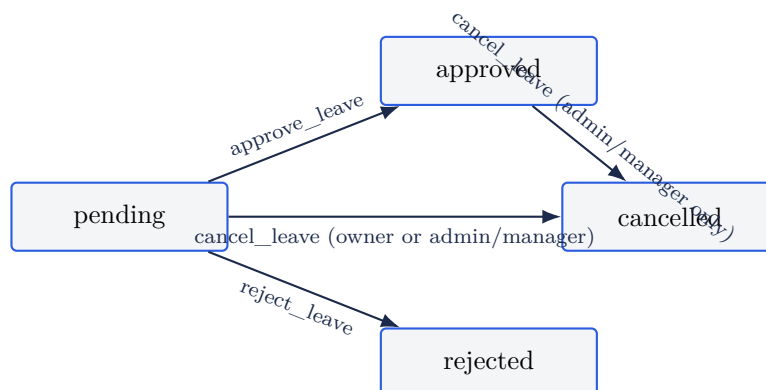
[PARTIALLY IMPLEMENTED]. There is no `BreakSession` table and no break start/end event anywhere in the schema. `break_minutes` is a single integer column on both `Shift` and `AttendanceEntry`, subtracted from the total duration when computing worked/scheduled hours (§4.7). There is no support for multiple breaks, break-start/end timestamps, or a "currently on break" status.

Edge Cases Explicitly Handled

- **Overnight entries** are a single row (not split at midnight); `business_date` attributes the whole entry to its start date.
- **Missing clock-out** is handled by the needs-review flag, never by inventing an end time.
- **Duplicate clock-in** is blocked by the DB's partial unique index, independent of any application-level check.
- **A closing time before the start time** is blocked by the `ended_after_started` CHECK (translated to a friendly error by `_flush_or_raise`).

4.5 Leave Domain

State Diagram



Only these four states exist (`status_valid` CHECK): `pending`, `approved`, `rejected`, `cancelled`. Notably, an *approved* request can still be cancelled (by admin/manager only, "the employee's plans changed") — the request's `decided_by_user_id/decided_at` are deliberately left untouched by `cancel_leave`, so an approved-then-cancelled request stays distinguishable from a pending-then-cancelled one using `status` alone (see the extended rationale in `app/models/leave_request.py`).

Approval and Conflict Detection

`leave.approve_leave` blocks the approval outright (with a `ValidationError` naming the specific conflicting shift ids and times) if any *published*, non-cancelled shift overlaps the requested range for that employee. Self-approval is also blocked: if the approving admin/manager happens to also be the requesting employee (compared via `scope.employee_id`), the approval is rejected. `leave.reject_leave` requires a mandatory `decision_note` (no such requirement for approval).

Authorization

An employee may create and cancel (while pending) only their own requests. Admin/manager may act on requests within their scope (any department for admin, managed departments for a manager), including approving/rejecting, and may cancel a request in *any* status.

Leave Balances

[PLANNED / NOT IMPLEMENTED]. `LeaveType` is a catalog only (name, whether it's paid, whether it requires approval, whether it blocks scheduling) — there is no ledger of days accrued or remaining anywhere in the schema or service layer.

4.6 Overtime Domain

Overtime is computed by `app.services.overtime`, a deliberately pure, database-free module (no I/O at all) so its tiering arithmetic can be unit-tested in complete isolation.

Configuration

An organization's overtime rules are effective-dated data (`OvertimePolicy` + child `OvertimeTier` rows), never hardcoded constants. A policy carries a daily threshold, a weekly threshold, and a `week_start_day`; its tiers (each scoped `daily` or `weekly`) carry a multiplier and an `[from_hours, to_hours)` range *beyond* the relevant threshold.

Calculation

Confirmed business rules (from the module's own docstring, not to be re-derived or second-guessed elsewhere in the codebase):

- Daily overtime: the first 2 hours beyond the daily threshold are paid at 1.5×; daily overtime beyond that (hour 3+ of daily OT) is paid at 2.0×.
- Weekly overtime: hours beyond the weekly threshold are paid at 1.5×, but *only* hours not already counted as daily overtime for that week — no worked hour is ever paid under two multipliers.
- No proration of the weekly threshold for a partial week.

For one day, given worked hours H , daily threshold T_d , and daily tiers $\{(f_i, t_i, m_i)\}$:

$$\text{regular_hours} = \min(H, T_d), \quad \text{overtime_hours} = \max(0, H - T_d)$$

`overtime_hours` is then split into per-tier buckets by `apply_tiers`, each bucket priced separately (§4.7). For a week, given each day's already-computed regular hours $r_1, \dots, r_7$ and weekly threshold T_w :

$$\text{weekly_overtime_eligible} = \max\left(0, \sum_{i=1}^7 r_i - T_w\right)$$

This is exactly what `compute_weekly_overtime` implements — summing *regular* hours (already net of daily OT), never total worked hours, is what prevents double-counting.

Effective Dates and Policy Resolution

`overtime.resolve_policy(organization_id, on_date)` finds the single policy whose `[effective_from, effective_to]` range covers `on_date` (enforced unique at the database level by `ex_overtime_policies_organization_no_overlap`), returning `None` if the organization has no policy configured for that date — callers must handle that case explicitly, never assume a default.

Reports and Authorization

`reports.overtime_summary` (admin/manager) reports per-employee overtime *hours* for a department/date range, reusing `labor_cost.range_cost_for_employee`'s already-correct tiering so an independent, hand-rolled hours calculation could never accidentally disagree with the money calculation. It reads only `LineItem.hours/.category` — never `.rate` or `.cost` — which is what makes this report safe for a manager to see under rule A4.

4.7 Labor Cost Domain

Labor cost is the money layer built on three already-verified components: worked seconds (`working_hours`), overtime tiering (`overtime`), and effective-dated Decimal pay rates (`pay_rates`). It adds no new business rule of its own — it only prices what those three already compute.

What "Labor Cost" Means Here

For each employee, each day, each priced category (regular time, a specific daily-OT tier, a specific weekly-OT tier), one `LineItem` is produced:

$$\text{cost} = \text{round}_{\text{HALF_UP}}(\text{hours} \times \text{rate} \times \text{multiplier}, 2\text{dp})$$

A department or range total is always the *sum of already-rounded line items*, never a rounding of one combined raw total — the two approaches can disagree by a cent, and the module's own test suite has a dedicated test proving they do in at least one real case.

Rounding

`Decimal` is used throughout (never `float`) specifically because floating-point binary representation cannot exactly represent most decimal fractions (e.g. \$0.10), which would silently accumulate rounding error across thousands of line items in a payroll-adjacent calculation — exactly the class of bug `.claude/rules/time-and-money.md` exists to prevent. Rounding uses `ROUND_HALF_UP` to two decimal places, applied once per line item.

Weekly Overtime Reclassification

The trickiest part of this module: a naive implementation would price an hour once as "daily regular" and again as "weekly overtime," overpaying. `range_cost_for_employee` instead *reclassifies* weekly-OT-eligible hours out of the regular pool *before* building the daily "regular" line items, walking backward from the end of the week (by convention, the hours that pushed the week over its threshold are the ones worked latest in it) so every worked hour is priced in exactly one line item. Each reclassified hour is priced at the rate in force on the specific day it actually came from — if an hourly rate changes mid-week, different portions of the same weekly-OT tier can legitimately be priced at different rates. The result is *prorated*: a weekly-OT line item is only emitted for the portion of reclassified hours whose source day actually falls inside the caller's requested date range, so that `cost(Mon..Sun) = cost(Mon..Wed) + cost(Thu..Sun)` for any adjacent split of a week — this exact invariant has a dedicated test.

Department Aggregation and What Each Role Can See

`labor_cost.department_cost_summary` (admin or manager, manager restricted to their own department) returns **only a total plus a count of unconfigured employees** — never a per-employee figure, never a rate. An employee with genuinely worked hours but no configured

pay rate or overtime policy for the range no longer blanks the whole department's total (a real bug fixed during development, called "Round A" in the module's docstring); instead they are excluded and counted, and the route surfaces that count as an explicit warning rather than a silently wrong total.

`labor_cost.range_cost_for_employee` (the underlying per-employee, per-line-item detail, including `.rate` and `.cost`) is only ever rendered by the admin-only `/labor-cost/employees/<id>` view — *no other route in the codebase calls this function and renders its rate/cost fields*, which is the actual mechanism (not just a convention) keeping this data away from managers.

Known, Documented Limitation: Paid Leave Is Not Costed

`LeaveType.is_paid` is stored on the schema but never read anywhere in `labor_cost.py` or elsewhere. Approved leave — paid or not — currently contributes exactly \$0 to every labor-cost figure in the system, because labor cost is derived solely from closed `AttendanceEntry` rows. This is documented explicitly in the module's own docstring as an open follow-up requiring a real business decision (does paid leave count toward overtime thresholds? at what rate is a leave hour priced?), not an oversight.

4.8 Reports

Report	Purpose	Filters	Source
Overtime report	Per-employee total overtime hours for a department/ range.	Department, start/end date	<code>reports.overtime_summary</code> (via <code>/reports/overtime</code>)
Hours trend	Per-day total worked hours for a department/range, shown as a bar comparison.	Department, start/end date	<code>reports.hours_trend</code> (via <code>/reports/hours-trend</code>)
Labor cost (department)	Aggregate labor cost total for a department/ range.	Department, start/end date	<code>labor_cost.department_cost_summary</code> (via <code>/labor-cost</code>)
Labor cost (employee, admin only)	Full per-line-item cost breakdown for one employee.	Employee (from URL), start/end date	<code>labor_cost.range_cost_for_employee</code> (via <code>/labor-cost/employees/<id></code>)

All four are department- or employee-scoped and date-range-filtered; none loads an unbounded table (the routes clamp an excessively wide `?start=/?end=` range to 92 days — see §6.2, security threat model, for why).

4.9 Audit Log

Every privileged or sensitive action in the system — login success/failure, pay rate changes, leave decisions, employee/department changes, shift publish/cancel, attendance corrections — is recorded through exactly one function, `audit.record`, which stages (but never itself commits) an `AuditLog` row. Every call site is required to let its own primary-write commit cover the audit entry too, so a crash between two separate commits can never leave a state change persisted with no audit trail.

The log is genuinely **append-only**: no route or service in the codebase ever issues an `UPDATE` or `DELETE` against `audit_logs`. This is enforced by convention (there is no database-level trigger preventing it), not by a database permission — **[NEEDS VERIFICATION]**

whether the production database role used by the application has been further restricted at the PostgreSQL grant level to make this literally impossible rather than merely unused by the current code.

`changes` is always a small, deliberately non-sensitive JSON summary (e.g. "a rate was set, for whom, effective when") — never a raw dump of the sensitive value itself. A pay-rate change never records the rate; a leave decision never records the free-text `decision_note` (which may contain a personal/medical circumstance).

Only an admin may read the log (`/audit-log`, paginated 50 rows at a time, date-range filtered) or its lighter "Recent Activity" variant (`/recent-activity`, unpaginated but capped, with actor emails resolved for readability) — enforced independently inside `audit.list_entries` and `reports.recent_activity` themselves, not just by the route decorator.

PART 5

Page-by-Page Documentation

Every substantive page in the application is documented below, in the order a user would encounter them. Error pages (400/403/404/413/500) are covered as a group at the end of this chapter rather than individually, since they share one template pattern and no business logic. Wireframes are simplified representations of the actual rendered layout (sidebar + topbar are omitted from most wireframes below the login page, since every authenticated page shares the same shell — see Chapter 6).

5.1 Login

Purpose: The only entry point for an unauthenticated visitor; establishes a session.

Route: GET/POST /login

Access: Public. An already-authenticated user is redirected to the dashboard instead of seeing the form again.

Main UI elements: Email field, password field, submit button, a flashed error banner on failure.

Data displayed: Nothing from the database — a static form.

User actions: Submit credentials.

Backend route: `auth.login (app/routes/auth.py)`

Service functions: `auth.service.authenticate`

Database data involved: `users` (read for the credential check and logout state; written on both success and failure to update `failed_login_count/locked_until/last_login_at`).

Authorization: None required to view; the route itself contains the only authorization logic in the system that runs *before* anyone is authenticated.

Validation: Both fields required (WTForms `DataRequired`); no email-format validation (deliberately — an invalid format simply won't match any account).

Edge cases: Wrong password, nonexistent email, inactive account, and locked account all show the identical generic error (Chapter 2). A successful login honors a same-site `?next=` redirect target and rejects anything else.

Error states: A flashed banner; the form is always safe to resubmit.

Related pages: Redirects successfully to the Dashboard.

Source files: `app/routes/auth.py`, `app/auth/service.py`, `templates/auth/login.html`

```
+-----+
| Workforce Management Platform |
| |
| [ Email ] |
```



```
| [ Password ] |
| [ Log in ] |
+-----+
```

5.2 Dashboard — Admin / Manager View

Purpose: "What is happening with the workforce right now" — the main operational landing page for admin/manager.

Route: GET /dashboard

Access: Admin (org-wide), manager (their managed departments).

Main UI elements: Metric grid (total employees, working today, absent today, on leave today), a "Requiring Attention" panel, detail tables (working/absent/on-leave/coverage), a labor-cost summary card, an optional department filter.

Data displayed: Headline counts; a list of attendance entries flagged `needs_review` and employees missing pay-rate/policy configuration; per-department coverage and labor-cost totals.

User actions: Filter by department; follow links into Attendance, Employees, or Labor Cost for detail.

Backend route: `dashboard.index`, delegating to `_manager_or_admin_dashboard` (`app/routes/dashboard.py`)

Service functions: `reports.who_is_working_today`, `reports.who_is_absent_today`, `reports.who_is_on_leave_today`, `reports.attendance_needing_review`, `labor_cost.department_cost_summary`

Database data involved: `shifts`, `attendance_entries`, `leave_requests`, `employees`, `employee_pay_rates`, `overtime_policies` (indirectly, via labor cost).

Authorization: `role_required` is not on this specific view function — `login_required` covers the whole `dashboard.index` route, which internally branches to the employee dashboard for an employee-role caller; every underlying `reports.*` call re-checks role/department scope itself.

Validation: An optional `?department_id=` is a plain integer filter; no format validation needed beyond that.

Edge cases: An employee with worked hours but no configured pay rate/overtime policy no longer blanks the whole department's cost total — they are excluded and counted explicitly (§4.6).

Error states: A `ValidationError` from the underlying labor cost calculation is flashed, not raised as a 500.

Related pages: Employees, Attendance, Schedule, Leave, Labor Cost, Reports — this page is the hub linking to all of them.

Source files: `app/routes/dashboard.py`, `templates/dashboard/admin.html`

```
+-----+
| Dashboard |
+-----+
| [Employees] [Working] [Absent] [On Leave] <- metrics |
+-----+
| Requiring Attention (needs-review entries, gaps) |
+-----+
| Working Today | Absent Today | On Leave | Coverage |
+-----+
| Labor Cost (department totals) |
+-----+
```

5.3 Dashboard — Employee View

Purpose: A personal "what's my status right now" landing page.

Route: GET /dashboard (same route, branches by role)

Access: Employee role only (own data).

Main UI elements: A prominent status card (Clock In / Currently Working / Attendance Needs Review), a metric grid (upcoming shifts, hours in the last 7 days, pending leave count), upcoming shifts table, recent hours table, leave requests table.

Data displayed: The caller's own shifts, attendance, and leave only.

User actions: Clock in / clock out; nothing else is writable from this page.

Backend route: `dashboard.index` → `_employee_dashboard`

Service functions: `reports.current_attendance_status`, `scheduling.list_shifts`, `working_hours.scheduled_vs_worked`, `leave.list_leave_requests`

Database data involved: `shifts`, `attendance_entries`, `leave_requests` — all filtered to `scope.employee_id`.

Authorization: Every underlying call is scoped to the caller's own `employee_id`; there is no parameter on this page that could target another employee.

Validation: None (no form inputs beyond the Clock In/Out buttons).

Edge cases: Three distinct states are rendered, not two: working, *needs review* (no button offered — see §4.4), or not clocked in. This distinction exists specifically to avoid presenting a Clock Out button that would always fail against a flagged entry.

Error states: A failed clock-in/out attempt (e.g. a race against the open-entry index) is flashed as a friendly message, never a raw 500.

Related pages: Attendance (full history), Schedule (full upcoming list), Leave.

Source files: `app/routes/dashboard.py`, `templates/dashboard/employee.html`

```
+-----+
| My Dashboard |
+-----+
| [ Currently Working / Not Clocked In ] [Clock Out/In] |
+-----+
| [Upcoming Shifts] [Hours 7d] [Pending Leave] |
+-----+
| Upcoming Shifts table | Recent Hours table |
+-----+
| My Leave Requests |
+-----+
```

5.4 Employees — List

Purpose: The organization's staff directory.

Route: GET/POST /employees

Access: Admin (all employees), manager (their departments). Not reachable by an employee-role user at all.

Main UI elements: Department/status filters, an "Add employee" disclosure (admin only), a table of employees with avatar-initial, name, employee number, department, status.

Data displayed: Every employee visible to the caller's scope.

User actions: Filter; create a new employee (admin only); click through to a detail page.

Backend route: `employees.list_employees / employees.create_employee`

Service functions: `employees.list_employees`, `employees.create_employee`, `departments.list_departments`

Database data involved: `employees`, `departments`.

Authorization: `role_required("admin", "manager")` on the route; `create_employee` additionally checks `scope.role == "admin"` inside the service.

Validation: Department/status filters are plain query parameters, filtered in Python over an already-scoped list (no route-level whitelist on the status value — [NEEDS VERIFICATION] whether this should reject an unrecognized status rather than silently returning no results).

Edge cases: A filtered view with zero matches is distinguished from a genuinely empty roster ("No employees match this filter" vs. "No employees yet").

Error states: An `IntegrityError` on create (duplicate employee number/email) is flashed, not a raw 500.

Related pages: Employee Detail, Departments, Schedule (assignment).

Source files: `app/routes/employees.py`, `templates/employees/list.html`

```
+-----+
| Employees [+ Add employee] |
+-----+
| Department [v] Status [v] [ Filter ] |
+-----+
| Employee | Department | Status |
| JS John Smith | Kitchen | Active |
+-----+
```

5.5 Employee Detail

Purpose: A single employee's full profile, plus (for admin/manager) a quick view of their schedule and attendance.

Route: `GET/POST /employees/<id>`

Access: Admin (any employee), manager (their departments), or the employee themselves (their own record only).

Main UI elements: Profile card (contact info, status, linked login account), an edit form (admin, or manager within scope), a terminate form (admin only), and — admin/manager only — Upcoming Shifts and Recent Attendance cards.

Data displayed: Employee profile fields; the linked `User` account's email/role/active status, if any; (admin/manager) the next 14 days of shifts and the last 7 days of attendance.

User actions: Edit profile; terminate (admin); navigate to Pay Rate or Labor Cost (admin only, via links in the page header).

Backend route: `employees.get_employee / employees.update_employee / employees.terminate_employee`

Service functions: `employees.get_employee`, `employees.get_linked_user`, `scheduling.list_shifts`, `reports.attendance_entries_with_context`

Database data involved: `employees`, `users`, `shifts`, `attendance_entries`.

Authorization: `employees.get_employee` 404s an employee attempting to view anyone but themselves; the Upcoming Shifts/Recent Attendance sections are computed and rendered only when `can_edit` is true (admin, or manager within scope) — an employee viewing their own page never sees them (that information already lives on their own dashboard).

Validation: Standard WTForms field validation on the edit form.

Edge cases: An employee with no linked login account shows "No login account linked" rather than a blank field.

Error states: A 404 for an out-of-scope employee id (never a 403, per the IDOR-defense convention in Chapter 2).

Related pages: Employees list, Pay Rate, Labor Cost breakdown, Schedule, Attendance.

Source files: `app/routes/employees.py`, `templates/employees/detail.html`

```
+-----+
| John Smith [Edit] [Pay rate] [Cost] |
+-----+
| Profile: number, department, status, email, account |
+-----+
| Upcoming Shifts table | Recent Attendance table |
+-----+
```

5.6 Employee Pay Rate

Purpose: View and set an employee's hourly-rate history.

Route: GET/POST `/employees/<id>/pay-rate`

Access: Admin only — never a manager, per rule A4.

Main UI elements: A history table (rate, effective from/to) and a "set new rate" form.

Data displayed: Every historical rate period for the employee.

User actions: Add a new effective-dated rate.

Backend route: `employees.view_pay_rate` / `employees.set_pay_rate`

Service functions: `pay_rates.list_pay_rate_history`, `pay_rates.set_pay_rate`

Database data involved: `employee_pay_rates`.

Authorization: `role_required("admin")` on the route, re-checked inside both service functions.

Validation: Rate must be > 0 ; effective-to (if given) must be on or after effective-from; the database's exclusion constraint is the final authority against an overlapping period.

Edge cases: An overlapping rate period is rejected with a friendly message, not a raw constraint-violation error.

Error states: `ValidationError` flashed on an invalid range or overlap.

Related pages: Employee Detail, Labor Cost (employee breakdown).

Source files: `app/routes/employees.py`, `app/services/pay_rates.py`, `templates/employees/pay_rate.html`

5.7 Departments

Purpose: Manage the organization's department list.

Route: GET/POST `/departments`, POST `/departments/<id>`, POST `/departments/<id>/deactivate`

Access: Admin (full CRUD + deactivate); manager (view only, scoped to managed departments).

Main UI elements: A table of departments (name, code, status), an edit form per row (admin), a create form (admin), a deactivate action (admin).

Data displayed: Every department visible to the caller.

User actions: Create, edit, deactivate (admin only).

Backend route: `departments.list_departments / create_department / update_department / deactivate_department`

Service functions: `departments.*` (all of them).

Database data involved: `departments`.

Authorization: List route is `role_required("admin", "manager")`; every write route is `role_required("admin")`, re-checked inside the service.

Validation: Name/code required; case-insensitive name uniqueness and (`organization_id`, `code`) uniqueness enforced at the database level.

Edge cases: Deactivation never deletes the row — employees and historical schedules may still reference it.

Error states: A duplicate name/code on create/update is flashed as "name or code already in use," not a raw constraint error.

Related pages: Employees, Schedule, Labor Cost.

Source files: `app/routes/departments.py`, `templates/departments/list.html`

5.8 Schedule

Purpose: The planned-work calendar.

Route: `GET/POST /schedule`, plus `/schedule/<id>`, `/schedule/<id>/assign`, `/schedule/<id>/publish`, `/schedule/<id>/cancel`

Access: Everyone can view (employees see only their own published shifts); admin/manager can create/edit/assign/publish/cancel.

Main UI elements: Date-range and department filters, a "New shift" disclosure, a table (date, start, end, hours, department, employee, status, actions).

Data displayed: Shifts in the selected window, scoped by role.

User actions: Create, edit (draft only), assign/reassign an employee, publish, cancel.

Backend route: `schedule.list_shifts` and the write actions listed above.

Service functions: `scheduling.*` (all of them).

Database data involved: `shifts`, `employees`, `departments`, and (for conflict checks) `leave_requests`.

Authorization: View is `login_required`; every write is `role_required("admin", "manager")`, with manager department scoping re-checked inside every `scheduling.*` write function.

Validation: End time after start time and a 24-hour span cap are database-enforced; overlap and approved-leave conflicts are checked before every create/reassign/time-edit.

Edge cases: An unassigned shift shows a warning badge, not silent blank text; a shift cannot be published without an assigned employee; an out-of-range date filter shows an explicit "invalid date range" state rather than a misleading "no shifts" message.

Error states: Overlap/leave-conflict violations are flashed as friendly, specific messages (Chapter 3's exclusion constraints, translated by `_commit_or_raise_overlap`).

Related pages: Employees, Leave (conflict source), Dashboard (coverage summary).

Source files: `app/routes/schedule.py`, `templates/schedule/list.html`

```
+-----+
| Schedule [+ New shift] |
+-----+
| From [ ] To [ ] Department [v] [ Filter ] |
```

```
+-----+
| Date Start End Hours Dept Employee Status Acts |
+-----+
```

5.9 Attendance

Purpose: The actual record of who worked when; the primary clock-in/clock-out surface.

Route: GET /attendance, POST /attendance/clock-in, POST /attendance/<id>/clock-out, POST /attendance/<id>/correct

Access: Everyone can view (employees see only their own entries); admin/manager can correct any entry in scope and clock in/out on behalf of others.

Main UI elements: A personal status card (employee only), a date-range filter plus an employee filter (admin/manager), a table (date, started, ended, status, worked hours, source, actions).

Data displayed: Attendance entries in the selected window, each enriched with computed worked-duration and lateness against a matched shift.

User actions: Clock in/out (self, or on behalf of others for admin/manager); correct an entry (admin/manager, mandatory reason).

Backend route: `attendance.list_entries` and the write actions listed above.

Service functions: `attendance.clock_in/clock_out/correct_entry`, `reports.attendance_entries_with_context`, `reports.current_attendance_status`

Database data involved: `attendance_entries`, `shifts` (for the matched-shift/lateness display).

Authorization: View is `login_required`; `correct_entry` is `role_required("admin", "manager")`; clock-in/out on someone else's behalf requires admin/manager, re-checked inside the service.

Validation: A backdated clock-in/out (admin/manager only) is capped at 90 days in the past and can never be in the future; a correction requires a non-blank reason.

Edge cases: A `needs_review` entry never offers a Clock Out button (it would always fail) — only a correction can resolve it; this status is detected by an unbounded lookup on `ended_at` IS NULL, not a recent-days window, specifically so an old, still-unresolved entry is never missed (see the note on this fix in Chapter 7).

Error states: A friendly message for a duplicate clock-in, an already-closed entry, or a needs-review clock-out attempt — never a raw 500 from the underlying database constraint.

Related pages: Dashboard (personal status card), Employee Detail (recent attendance), Schedule (matched shift source).

Source files: `app/routes/attendance.py`, `templates/attendance/list.html`

```
+-----+
| Attendance |
+-----+
| [ Currently Working / Not Clocked In ] [Clock Out/In] |
+-----+
| From [ ] To [ ] Employee [v] [ Filter ] |
+-----+
| Date Started Ended Status Worked Source Actions |
+-----+
```

5.10 Leave

Purpose: Request and manage time off.

Route: GET/POST /leave, plus approve/reject/cancel actions

Access: Everyone can view (employees see only their own requests) and request leave; admin/manager can approve/reject/cancel within scope.

Main UI elements: Status and employee filters (admin/manager), a request form, a table (dates, employee, type, status, reason, actions), inline conflict warnings.

Data displayed: Leave requests in scope; for a pending request, any conflicting published shift.

User actions: Request leave; approve/reject (with a mandatory reason for rejection); cancel.

Backend route: `leave.list_requests` and the write actions.

Service functions: `leave.*` (all of them), `leave.conflicting_shifts_for`

Database data involved: `leave_requests`, `leave_types`, `shifts` (conflict detection).

Authorization: View/request is `login_required`; approve/reject is `role_required("admin", "manager")`; cancel is open to the owner (while pending) or admin/manager (any status), enforced inside `cancel_leave` itself.

Validation: Rejection requires a non-blank reason; an overlapping pending/approved request for the same employee is rejected by the database's exclusion constraint.

Edge cases: Self-approval is explicitly blocked, even for an admin/manager who is also the requesting employee; approving over a published shift is blocked with the specific conflicting shift(s) named.

Error states: A friendly, specific conflict message rather than a generic failure.

Related pages: Schedule (conflict source), Dashboard ("on leave today").

Source files: `app/routes/leave.py`, `templates/leave/list.html`

5.11 Overtime Report

Purpose: Per-employee overtime hours for a department/date range.

Route: GET /reports/overtime

Access: Admin, manager (their departments).

Main UI elements: Department + date-range filter, a table (employee, overtime hours), a total row, cross-navigation to Hours Trend and Labor Cost for the same filter.

Data displayed: Overtime *hours* only — never a rate or a cost figure (rule A4).

User actions: Filter; navigate to related reports.

Backend route: `dashboard.overtime_report`

Service functions: `reports.overtime_summary`

Database data involved: `attendance_entries`, `overtime_policies/overtime_tiers`, `employee_pay_rates` (read internally to price line items, never rendered here).

Authorization: `role_required("admin", "manager")`, with manager department scoping re-checked inside the service.

Validation: An excessively wide date range is clamped to 92 days before the underlying per-employee/per-day calculation runs (Chapter 6).

Edge cases: An employee with no configured pay rate/policy shows "Not available" for that row instead of failing the whole report.

Error states: None beyond the per-row "not available" case — the report always renders.

Related pages: Hours Trend, Labor Cost, Dashboard.

Source files: `app/routes/dashboard.py`, `templates/reports/overtime.html`

5.12 Hours Trend Report

Purpose: "Are working hours going up or down over time" for a department.

Route: GET `/reports/hours-trend`

Access: Admin, manager (their departments).

Main UI elements: Department + date-range filter, a per-day bar comparison of total worked hours, cross-navigation to Overtime and Labor Cost.

Data displayed: Total worked hours per day, summed across the department's employees.

User actions: Filter; navigate to related reports.

Backend route: `dashboard.hours_trend_report`

Service functions: `reports.hours_trend`

Database data involved: `attendance_entries`.

Authorization: `role_required("admin", "manager")`.

Validation: Same 92-day clamp as the Overtime report; an `end < start` range shows an explicit "invalid date range" state.

Edge cases: Bar width is computed server-side and mapped onto a small set of CSS width classes (never an inline `style` attribute, to respect the application's strict CSP — see Chapter 6).

Error states: None beyond the invalid-range state.

Related pages: Overtime Report, Labor Cost, Dashboard.

Source files: `app/routes/dashboard.py`, `templates/reports/hours_trend.html`

5.13 Labor Cost — Department Totals

Purpose: Aggregate labor cost for a department/date range.

Route: GET `/labor-cost`

Access: Admin, manager (their departments).

Main UI elements: Department + date-range filter, a total-cost card, an unconfigured-employee warning, cross-navigation to Hours Trend/Overtime, and (admin only) a per-employee drill-down list linking to individual breakdowns.

Data displayed: A single aggregate Decimal total — never a rate, never a per-employee figure, for a manager.

User actions: Filter; (admin) drill into one employee's breakdown.

Backend route: `labor_cost.department_totals`

Service functions: `labor_cost.department_cost_summary`

Database data involved: `attendance_entries`, `employee_pay_rates`, `overtime_policies/tiers` (internally).

Authorization: `role_required("admin", "manager")`; the per-employee drill-down section is computed and rendered only when `is_admin` — a manager never sees even the *existence* of that link, not just its numbers.

Validation: Same 92-day range clamp as the other reports; `end < start` raises a `ValidationError`, flashed rather than crashing.

Edge cases: The per-employee drill-down list includes *every* employment status (not only active), matching exactly the employee set the total above it was computed from — otherwise the total and the list of links would silently disagree about who contributed to it.

Error states: A flashed message on an invalid range; a "no cost total recorded" state when a valid department/range yields no data.

Related pages: Overtime Report, Hours Trend, Labor Cost Employee Detail (admin only).

Source files: `app/routes/labor_cost.py`, `templates/labor_cost/index.html`

5.14 Labor Cost — Employee Breakdown

Purpose: The one view in the system that shows an individual employee's hourly rate and exact cost, broken down by day and category.

Route: `GET /labor-cost/employees/<id>`

Access: Admin only — structurally unreachable by a manager (hard-coded `role_required("admin")`, not `role_required("admin", "manager")` like almost every other manager-adjacent view).

Main UI elements: Date-range filter, a line-item table (date, category, hours, rate, multiplier, cost), a total row.

Data displayed: Every priced line item for the employee/range, including the hourly rate in force on each day.

User actions: Filter by date range.

Backend route: `labor_cost.employee_detail`

Service functions: `labor_cost.range_cost_for_employee`

Database data involved: `attendance_entries`, `employee_pay_rates`, `overtime_policies/tiers`.

Authorization: `role_required("admin")`, plus an org-scoped 404 lookup on the employee id (defense in depth, since the route is already admin-only).

Validation: Same range clamp; a day with genuinely worked hours but no configured rate/policy raises a specific `ValidationError`, flashed rather than silently priced as zero.

Edge cases: A mid-week rate change can legitimately produce two differently-priced weekly-overtime line items for the same tier (§4.6).

Error states: A flashed configuration-gap message; the table simply shows whatever line items *did* resolve.

Related pages: Employee Detail, Pay Rate, Labor Cost Department Totals.

Source files: `app/routes/labor_cost.py`, `templates/labor_cost/employee_detail.html`

5.15 Audit Log

Purpose: A complete, permanent, admin-only record of privileged actions.

Route: `GET /audit-log`

Access: Admin only.

Main UI elements: A date-range filter, a paginated table (when, action, entity, actor user id, IP address, details), Previous/Next links.

Data displayed: Every audit entry in the selected window, 50 per page.

User actions: Filter by date; page through results.

Backend route: `audit.list_entries`

Service functions: `audit.list_entries`

Database data involved: `audit_logs`.

Authorization: `role_required("admin")`, re-checked inside `audit.list_entries` itself (this route historically queried the database directly; it now goes through the service like every other route — see Chapter 1's "Round C fix" note).

Validation: Page number is clamped to a minimum of 1; an invalid date string falls back to the default window rather than erroring.

Edge cases: The actor's raw user id is shown, not a resolved email — unlike the Recent Activity view below — **[NEEDS VERIFICATION]** whether this asymmetry is intentional or an omission.

Error states: An empty range shows a clear "no audit events in this range" state.

Related pages: Recent Activity (a lighter view over the same data).

Source files: `app/routes/audit.py`, `app/services/audit.py`, `templates/audit/list.html`

5.16 Recent Activity

Purpose: A lighter, easier-to-read glance at the most recent privileged actions, for a quick check rather than a full investigation.

Route: `GET /recent-activity`

Access: Admin only.

Main UI elements: A date-range filter, an activity list with each entry's actor email resolved, a link through to the full Audit Log.

Data displayed: The same underlying audit events as the Audit Log, capped at `audit.DEFAULT_PAGE_SIZE` (50) with no pagination controls.

User actions: Filter by date; follow through to the full log.

Backend route: `audit.recent_activity`

Service functions: `reports.recent_activity` (which itself calls `audit.list_entries` and resolves actor emails in one batched query).

Database data involved: `audit_logs`, `users` (actor email resolution).

Authorization: `role_required("admin")`, re-checked inside `reports.recent_activity` itself.

Validation: Same date-parsing fallback as the Audit Log.

Edge cases: An entry whose actor account no longer resolves (should not normally happen, since users are never hard-deleted) shows no email rather than erroring.

Error states: Same "no events" empty state pattern as the Audit Log.

Related pages: Audit Log.

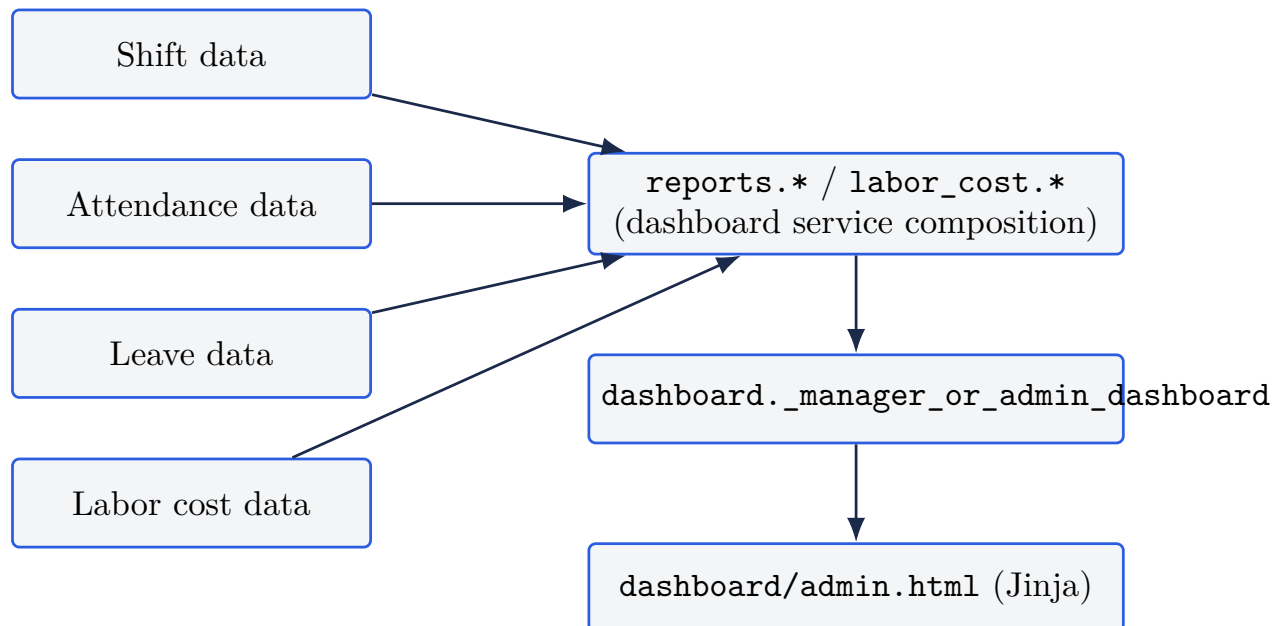
Source files: `app/routes/audit.py`, `app/services/reports.py`, `templates/audit/recent_activity.html`

5.17 Error Pages

Five static error templates exist, one per handled HTTP status: `errors/400.html`, `403.html`, `404.html`, `413.html` (payload too large — reachable because `MAX_CONTENT_LENGTH` is set to 1 MB), and `500.html`. None of them ever renders a stack trace, a SQL error, or any other internal detail — `app.errors.handle_internal_server_error` logs the real exception server-side and returns only a generic message. None require authorization of their own (a 404 is deliberately returned *instead of* a 403 for an out-of-scope resource, per Chapter 2's IDOR defense, so these pages are reached by every role equally).

5.18 Admin Dashboard Deep Dive

The Admin Dashboard (§5.2) is the most information-dense page in the application, so its data flow is documented separately here.



Metrics. Total employees, currently working, absent, on leave — each a thin composition over an existing `reports.who_is_*` function, not a new query.

Workforce status / attendance. `who_is_working_today` and `who_is_absent_today` together define "absent" as *scheduled but with no attendance entry at all today, and not on approved leave* — not "did not clock into the specific shift they were assigned," a distinction that matters for an employee with two shifts or an early clock-in outside the shift-matching grace window (documented in `reports.py`'s "Round B fix" comment).

Departments / coverage / labor cost. Computed once per visible department via `scheduling.coverage_summary` and `labor_cost.department_cost_summary`.

Recent activity. Available as its own page (§5.15), reached from the sidebar, not embedded on the dashboard itself.

Filters. An optional department filter narrows every section above simultaneously.

Admin-only vs. manager-safe information. Everything on this page is manager-safe *except* that a manager only ever sees their own managed departments' data (the query scoping, not a separate rendering path, is what enforces this — there is no "admin-only widget" hidden from a manager on this particular page, unlike the Labor Cost and Pay Rate pages).

PART 6

Frontend Architecture and Security

6.1 Frontend Architecture

Templates

Every page is rendered server-side with Jinja2. `templates/base.html` defines the one shared page skeleton: a skip link, the authenticated application shell (sidebar + topbar + `<main>`) when `current_user.is_authenticated`, or a bare centered card otherwise (login, error pages). Every other template extends `base.html` and fills in `{% block title %}`, `{% block page_header %}`, and `{% block content %}`.

Shared, non-page markup lives in `templates/components/`: `icons.html` (a small inline-SVG icon set — no external icon font, since the CSP allows no CDN), `badges.html` (a single `status_badge(value, variant_map, label_map)` macro used by every status column in the application, so the status-to-color mapping exists in exactly one place), `macros.html` (`nav_group` / `nav_link`), `sidebar.html`, and `topbar.html`.

CSS Architecture

Four files, loaded in a fixed order (`base.html`'s `<head>`):

1. `tokens.css` — design tokens only (see §6.2).
2. `base.css` — element resets, typography defaults, focus states, the `.sr-only` utility.
3. `components.css` — reusable component classes (`.btn-*`, `.card`, `.badge-*`, `.table-wrap`, `.state-card`, `.nav-link`, `.status-card`, ...).
4. `layout.css` — the application shell grid, sidebar drawer behavior, and responsive breakpoints.

No page defines its own one-off styles outside this system — every page-specific need (Attendance's "Worked" column badge, Labor Cost's total line) is expressed by composing existing component classes.

JavaScript

`static/js/main.js` is deliberately small and contains **no business logic** — everything it does is browser presentation behavior, per `.claude/rules/frontend.md`:

- The mobile/tablet sidebar drawer: open/close, an overlay, Escape to close, and — notably — applying the HTML `inert` attribute to whichever side of the layout is not currently interactive, so keyboard Tab navigation can never reach off-canvas or dimmed-behind-overlay content. This is a genuine accessibility feature, not just a visual toggle.
- Two small, generic, CSP-safe behaviors driven by data attributes rather than inline event handlers (which the application's strict CSP silently drops in every real browser): `[data-autosubmit]` submits a form when a `<select>` changes (with a real submit button always present as a fallback), and `[data-confirm="..."]` shows a native `confirm()` dialog

before a destructive form submits — purely client-side friction; the server remains authoritative regardless.

Components

Reusable UI is expressed as Jinja macros/includes, not JavaScript components — there is no client-side component framework at all. This matches `CLAUDE.md`'s explicit instruction not to introduce React/Vue/Angular without justification.

Responsive System

Two shell-level breakpoints (`layout.css`): 768px and 1024px, controlling when the sidebar collapses into the off-canvas drawer described above. Individual components add their own narrower breakpoints where needed (720px, 480px, 420px in `components.css`, 900px for a filter-bar layout change) — responsive behavior is handled per-component, not as one global "mobile mode."

Accessibility

- A skip-link (`<a class="skip-link" href="#main-content">`) as the very first focusable element on every page.
- `:focus-visible` styling on every interactive element (`base.css`) — focus outlines are never removed without a replacement.
- `@media (prefers-reduced-motion: reduce)` disables non-essential transitions.
- Sidebar navigation group labels ("Workforce", "Operations", ...) are visually hidden (`.sr-only`) but still announced to screen readers via the group `<ul>`'s `aria-labelledby` — a sighted user sees per-link icons instead of a group heading, but the semantic structure a screen reader relies on is unchanged.
- The mobile drawer's focus management (described above) is a genuine focus-trap-equivalent implementation using `inert`, not just a CSS `display: none`.

6.2 Design System

Defined entirely as CSS custom properties in `tokens.css`.

Typography. Two self-hosted variable fonts: Figtree (body/UI text, buttons, tables) and Hanken Grotesk (headings, the brand wordmark, large emphasized figures like metric values). Both are served from `static/fonts/` as `woff2` — not a Google Fonts `<link>` — specifically so the strict CSP needs no `font-src` exception at all.

Colors. HSL channel triplets (e.g. `-ink: 222 32% 14%`) rather than full `hsl(...)` values, so any rule can pull an alpha variant via `hsl(var(-x) / 40%)` — this is what produces the whole "ink at N% opacity" hierarchy from a handful of base tokens instead of a long list of hand-picked greys. Semantic colors (`-success`, `-warning`, `-danger`, `-info`, `-done`) are reserved for real system states, never decoration, and their exact lightness values were tuned to clear WCAG's 4.5:1 contrast ratio as badge text (documented in the file's own comments, including two real failures — 4.1–4.3:1 and 3.05:1 — caught and fixed before shipping).

Theming. Three states: an unstamped root is the light palette; an OS-level dark preference (`prefers-color-scheme: dark`) redefines the tokens, guarded so an explicit `data-theme="light"` still wins; an explicit `data-theme="dark"` redefines them again so an in-app toggle wins in both directions.

Spacing, radius, motion. A 4px-based spacing scale (`-sp-1` through `-sp-24`), two radius tokens (`-radius-control` for buttons/inputs, `-radius-lg` for containers), and a single `-transition-fast` (120ms) used for every micro-interaction.

Components documented by their own CSS comments. Buttons (`.btn-primary` / `.btn-secondary` / `.btn-danger`), forms, tables (`.table-wrap`), cards (`.card`, `.summary-card`), status badges (`.badge-*`, mapped per-domain by `badges.html`'s macro), metric cells (`.metric-grid` / `.m-value`), and navigation (`.nav-link`, with an `::before` left-edge accent bar for the active item) — all styled from the same token set, so changing the accent color is a one-line edit in `tokens.css`, not a find-and-replace across the codebase.

6.3 Security Architecture

Every mechanism below was verified directly against the code that implements it, per this document's own accuracy requirement — nothing here is claimed "secure" merely because Flask or a library nominally supports it.

Authentication & Password Hashing

Covered in full in Chapter 2, §2.3: Argon2 hashing, generic error messages, timing-attack mitigation via a dummy-hash verification on every failure path, and a 5-attempt/15-minute lockout policy.

Session Security

`SESSION_COOKIE_HTTPONLY = True` (always) and `SESSION_COOKIE_SAMESITE = "Lax"` (always) prevent, respectively, client-side script access to the cookie and most cross-site request delivery. `SESSION_COOKIE_SECURE` is `False` in development/testing (there is no HTTPS to require) and `True` in `ProductionConfig`. `PERMANENT_SESSION_LIFETIME` (12 hours) bounds how long a stolen or forgotten-open session cookie remains valid, independent of activity. Login also calls `session.clear()` before establishing the new session, discarding any pre-authentication state (session-fixation defense).

Authorization & Access Scopes

The whole of Chapter 2 — `AccessScope`, defense-in-depth, `get_scoped_or_404`'s 404-not-403 IDOR defense, and the manager/admin/employee boundaries — applies here directly and is not repeated.

CSRF Protection

Flask-WTF's `CSRFProtect` is initialized globally (`app.extensions.csrf`), so every state-changing POST route requires a valid CSRF token by default. Every form in `app/forms.py` is a `FlaskForm`, and every template renders `{{ form.hidden_tag() }}` (or a manually-included `csrf_token()` hidden field for the handful of button-only forms with no `WTForms` class, e.g. the logout form in `topbar.html`). `TestingConfig` disables CSRF (`WTF_CSRF_ENABLED = False`) purely so route tests can post form data without simulating a token fetch first — this setting is not reachable in development or production.

Input Validation

Two layers: `WTForms` validators at the form boundary (`DataRequired`, `Length`, `NumberRange`, `Optional`) reject malformed input before a route ever calls a service; every service function then re-validates the business meaning of its inputs (an employee id that doesn't exist in this organization, a date range where `end < start`, a rate that isn't positive) regardless of what the form already checked, raising a typed `ValidationError` the route flashes back to the user.

SQL Injection Protection

Every database query in the codebase is built through SQLAlchemy’s query API (`db.session.query(...).filter(...)`) — there is no raw, string-interpolated SQL anywhere in `app/`, verified by searching for `execute()` and hand-built SQL strings. The few uses of SQLAlchemy’s `text(...)` helper are DDL-time expressions inside model `__table_args__` (a functional index expression, and the `'[]'` range-bounds literal for `daterange/tstzrange` exclusion constraints) — fixed strings written by the developer, never built from request input.

XSS Protection

Jinja2’s autoescaping is on by default and is never bypassed: a search of every template confirmed there is no `|safe` filter, no `{% autoescape false %}`, and no `Markup(...)` call anywhere in the codebase. Every value interpolated into HTML — an employee’s name, a leave request’s free-text reason, an audit log’s JSON `changes` — is escaped by the template engine before it reaches the browser.

Content Security Policy

Every response carries:

```
default-src 'self'; frame-ancestors 'none'; base-uri 'none';  
form-action 'self'; object-src 'none'
```

This is affordable specifically because the application serves no inline scripts, no external scripts/styles/fonts, and no embeddable content — every design decision in §6.1–6.2 above (self-hosted fonts, inline SVG icons instead of an icon font, data-attribute-driven JS instead of inline handlers) exists in service of this one policy holding without exceptions.

Other Security Headers

Attached to every response by one `@app.after_request` hook: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, and — conditionally, only when `SESSION_COOKIE_SECURE` is true, so a plain-HTTP development server never sends it — `Strict-Transport-Security: max-age=31536000; includeSubDomains`.

Secure Cookies & Cache Control

Covered under Session Security above. **[NEEDS VERIFICATION]**: no explicit `Cache-Control` header is set on authenticated pages containing potentially sensitive data (labor cost, pay rates); this is not verified as either a real gap or a non-issue for this application’s deployment model.

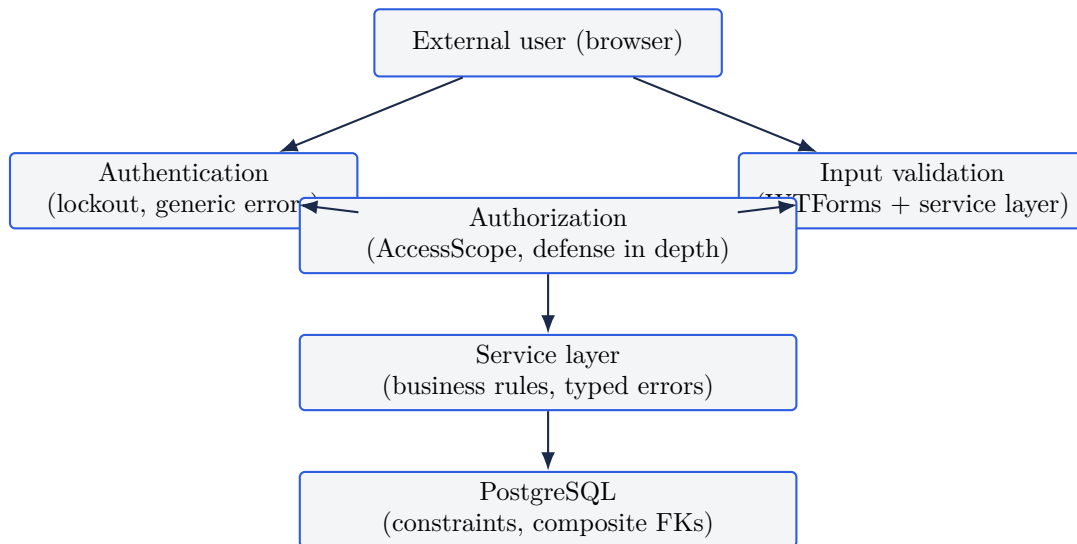
Production Configuration

`ProductionConfig.__init__` raises `RuntimeError` immediately if `SECRET_KEY` or `DATABASE_URL` is missing from the environment — the application cannot start in an insecure default state. An unrecognized `FLASK_ENV` value also raises rather than silently falling back to `DevelopmentConfig` (which would otherwise mean `DEBUG=True` and no secure cookies/HSTS by mistake).

Error Handling & Sensitive Information Exposure

Every registered error handler (§1.4, §5.16) renders a generic template; the 500 handler specifically logs the real exception server-side via Python’s `logging` module and never includes it in the response. `MAX_CONTENT_LENGTH` (1 MB) makes the registered 413 handler reachable and bounds request body size generally. `.env` (containing `DATABASE_URL/SECRET_KEY`) is listed in `.gitignore` and was not found committed to the repository.

6.4 Security Threat Model



Realistic threats this design specifically defends against:

- **A manager probing another department’s data by editing a URL/query parameter (IDOR).** Mitigated by `get_scoped_or_404` and every service function’s own scope filter, returning 404 rather than leaking existence via a 403. Directly tested by `tests/routes/test_idor_sweep.py` and the manager-scoped filter tests added alongside the `?employee_id=?department_id=` filters described in Chapter 7.
- **Credential stuffing / brute force against login.** Mitigated by the 5-attempt lockout and Argon2’s inherent cost per guess.
- **Email enumeration via login timing.** Mitigated by the dummy-hash verification on every non-success path.
- **Session fixation / cookie replay.** Mitigated by `session.clear()` before login, `HttpOnly/Secure/SameSite` cookie flags, and an absolute 12-hour session lifetime.
- **Cross-site request forgery** on any state-changing action. Mitigated by Flask-WTF’s CSRF token on every form.
- **Reflected/stored XSS** via a free-text field (a leave reason, an attendance note, an audit changes value). Mitigated by Jinja’s default autoescaping, never bypassed.
- **An unbounded report query as a self-inflicted denial of service** (a caller-supplied `?start=?end=` spanning decades against a per-employee/per-day calculation loop). Mitigated by a 92-day range clamp on the Overtime, Hours Trend, and Labor Cost report routes.

- **A manager inferring an individual’s pay rate from an aggregate total.** Explicitly *not fully preventable* and documented as such in `labor_cost.department_cost_summary`’s own docstring: the function guarantees it never *directly* returns a rate or per-employee figure, but cannot stop a manager who separately knows one employee’s hours for a range from dividing the total to estimate a rate. The real mitigation is workflow-level (department-wide reporting is the intended use), not a technical guarantee.

Explicitly out of scope for this threat model (not present in the codebase, and not claimed to be): infrastructure-level threats (network segmentation, WAF, DDoS protection), client-device compromise, and social-engineering attacks against a legitimate admin’s own credentials.

PART 7

Testing, Data Flows, and Business Rules

7.1 Testing Architecture

458 tests across 43 files, organized by *kind* rather than by feature:

```
tests/
|-- unit/ (4 files) Pure functions, no DB, no Flask app
| |-- test_overtime.py Tiering math in isolation
| |-- test_passwords.py Argon2 wrapper
| |-- test_redirects.py Open-redirect guard
| '-- test_access_scope.py AccessScope construction
|-- integration/ (23 files) Service layer against a real test DB
| |-- test_*_service.py One per service module
| |-- test_*_constraints.py Database CHECK/EXCLUDE/UNIQUE behavior
| '-- test_overtime_resolver.py Effective-dated policy lookup
|-- routes/ (14 files) Full HTTP request/response cycle
| |-- test_*_routes.py One per blueprint
| |-- test_idor_sweep.py Cross-tenant/cross-department 404s
| |-- test_decorators.py login_required / role_required
| |-- test_security_headers.py Response headers on every route
| '-- test_manager_with_no_departments.py A real edge case, not a mock
'-- (top-level: test_app_factory.py, test_database_connection.py)
```

Unit tests need no database or Flask application context at all — `tests/unit/test_overtime.py` is the clearest example: the entire daily/weekly tiering algorithm is exercised as plain Python function calls with `Decimal` inputs.

Integration tests exercise a service function against a real PostgreSQL database (a dedicated test database, configured via `TEST_DATABASE_URL`), inside a transaction that is rolled back after each test (`db_session` fixture in `tests/conftest.py`). The `_constraints.py` files specifically target database-level behavior — an `IntegrityError` from a `CHECK/EXCLUDE/UNIQUE` constraint — independent of any application code path that might also enforce the same rule.

Route tests drive the application exactly as an HTTP client would: log in via `POST /login` with real form data (CSRF disabled in `TestingConfig`), then issue further requests against protected routes, asserting on status codes and response body content.

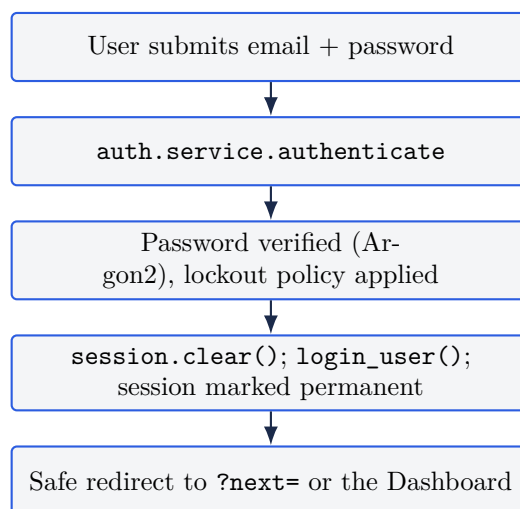
7.2 Important Test Cases

- `test_idor_sweep.py` — drives write routes directly with an id belonging to another organization or an unmanaged department, and asserts a plain **404**. This is the single most important file in the suite for the authorization model in Chapter 2: it protects against the exact class of bug (an id-based lookup that forgets to filter by scope) that a unit test of the "happy path" would never catch.

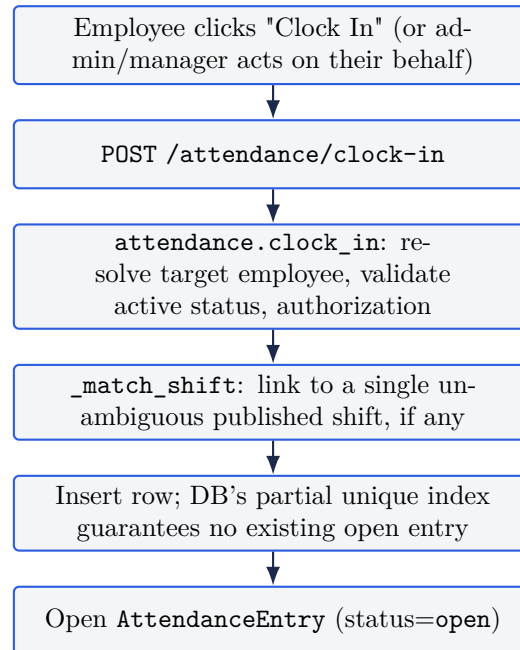
- **test_auth_service_timing.py** (`TestLoginTimingParity`) — protects the anti-enumeration timing defense (§2.3/§6.2) by asserting the dummy-hash verification actually runs on every failure branch.
- **test_attendance_constraints.py** — protects the duplicate-clock-in guarantee (partial unique index), the overlap exclusion constraint, the 24-hour duration cap, and the status/ended_at nullability CHECK, each as a direct database-level test independent of the service layer.
- **test_overtime_constraints.py** / **test_overtime_resolver.py** — protect the effective-dating exclusion constraint (no two overlapping policies for one organization) and correct policy resolution among several historical ones.
- **test_overtime.py** (unit) — protects the pure tiering math (`TestApplyTiers`, `TestComputeDailyOvertime`, `TestComputeWeeklyOvertime`) with no database involved at all, so a regression here is caught in milliseconds.
- **test_labor_cost_service.py** — includes a dedicated test proving line-item rounding and total-of-rounded-items can diverge from rounding-a-combined-total by a cent (§4.6), and a dedicated proration invariant test ($\text{cost}(\text{Mon..Sun}) = \text{cost}(\text{Mon..Wed}) + \text{cost}(\text{Thu..Sun})$).
- **test_manager_with_no_departments.py** — a real edge case (a manager account with zero `department_managers` rows) exercised end-to-end across the dashboard, department list, and employee list, rather than assumed to be handled correctly.
- **Route tests added for this documentation pass's own recent work** — e.g. `test_employees_own_needs_review_entry_shows_no_clock_out_button` and `test_needs_review_entry_older_than_a_day_still_shows_attention_state` in `test_attendance_routes.py` protect the "current status" lookup fix described in §7.5.

7.3 Important System Flows

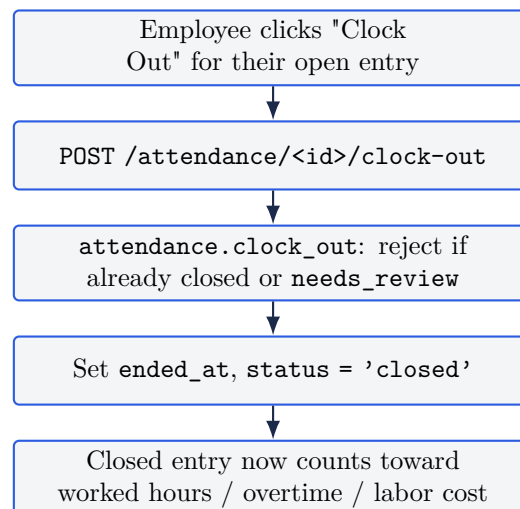
Login Flow



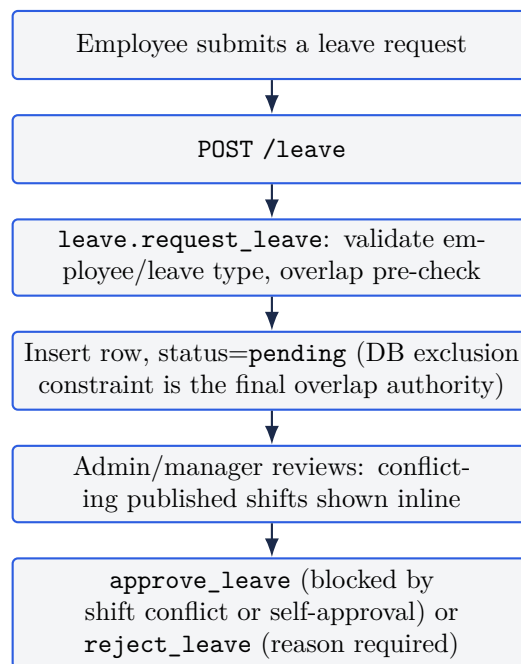
Check-In Flow



Check-Out Flow



Leave Request Flow



7.4 Business Rules Catalog

BR-001 — Attribution date follows the organization's local timezone (rule A1)

Description: A shift's or attendance entry's `business_date` is the local calendar date of its start time in the organization's own timezone, not the server's — an overnight period is attributed entirely to its start date, never split.

Where implemented: `scheduling.business_date_for`, reused by `attendance.clock_in/correct_entry`.

Why it exists: A server in UTC and an organization in, say, Pacific/Auckland disagree about "today" for roughly half of every day; every "as of today" report must agree on one definition.

Tests protecting it: `test_attendance_constraints.py`'s `test_overnight_entry_is_stored_as_a_single_row_with_start_date_attribution`; `test_default_date_range_uses_the_organizations_local_date_not_the_servers` in `test_schedule_routes.py`.

BR-002 — Shift-match grace window (rule A3)

Description: A clock-in within 60 minutes of a published shift's start or end still counts as "working that shift" for automatic matching purposes.

Where implemented: `attendance._match_shift(_SHIFT_MATCH_GRACE)`.

Why it exists: Employees rarely clock in at the exact minute a shift starts; without a grace window, "who is absent today" would misclassify a slightly-early or slightly-late arrival as unscheduled work with no matched shift.

Tests protecting it: Attendance service integration tests covering shift matching.

BR-003 — Managers never see an individual pay rate or per-employee cost (rule A4)

Description: A manager may see department-level cost *totals* and per-employee overtime *hours*, but never an hourly rate or an individual's exact cost.

Where implemented: `role_required("admin")` (not `"admin"`, `"manager"`) on the `pay-rate` and `employee-cost-breakdown` routes; `reports.overtime_summary` reads only `.hours/.category`.

Why it exists: Pay rates are more sensitive than aggregate cost figures; a manager needs enough information to manage a budget, not to know a colleague's exact wage.

Tests protecting it: `test_labor_cost_routes.py`'s `manager-cannot-reach-admin-only-detail` tests and the `Per-Employee-Breakdown` visibility test; `test_dashboard_routes.py`'s `test_manager_never_sees_pay_rate_or_cost_on_overtime_report`.

BR-004 — Maximum 24-hour span for a shift or attendance entry (rule A7)

Description: No single shift or attendance entry may span more than 24 hours.

Where implemented: `duration_max_24_hours` CHECK constraints on both shifts and attendance entries.

Why it exists: Without a cap, a colluding manager and employee (or a data-entry mistake) could post an arbitrarily long entry that flows straight into overtime/labor-cost calculations.

Tests protecting it: `test_attendance_constraints.py`'s `test_duration_max_24_hours_check_rejects_an_overlong_entry`.

BR-005 — Stale open entries are flagged, never auto-closed (rule A11)

Description: An open attendance entry with no clock-out after a configurable cutoff (default 16 hours) is marked `needs_review` — the system never invents an end time for it.

Where implemented: `attendance.flag_stale_open_entries`, run via flask `attendance flag-stale`.

Why it exists: Guessing an end time would silently fabricate paid hours; flagging for human review preserves data integrity at the cost of requiring a correction.

Tests protecting it: Attendance service tests covering `flag_stale_open_entries`; `TestAttendanceNeedingReview` in `test_reports_service.py`.

BR-006 — A needs-review entry can only be resolved by a correction, not a plain clock-out

Description: `clock_out` explicitly rejects an entry in `needs_review` status; only `correct_entry` (admin/manager, mandatory reason) can resolve one.

Where implemented: `attendance.clock_out`.

Why it exists: Allowing a plain clock-out to resolve a flagged entry would defeat the entire purpose of flagging it — there would be no cap on the resulting duration and no human review at all.

Tests protecting it: `test_employees_own_needs_review_entry_shows_no_clock_out_button` and `test_needs_review_entry_older_than_a_day_still_shows_attention_state` (`test_attendance_routes.py`).

BR-007 — No worked hour is priced under two overtime multipliers

Description: Weekly overtime is computed only from hours not already counted as daily overtime for that week.

Where implemented: `overtime.compute_weekly_overtime`; the reclassification logic in `labor_cost.range_cost_for_employee`.

Why it exists: A naive implementation would pay the same hour once at a daily multiplier and again at a weekly one, silently overpaying — explicitly forbidden.

Tests protecting it: `TestComputeWeeklyOvertime` (`tests/unit/test_overtime.py`); the proration invariant test in `test_labor_cost_service.py`.

BR-008 — Money is rounded per line item, never from a combined total (rule A12)

Description: Each priced `LineItem` is independently rounded (`ROUND_HALF_UP`, 2dp); a total is the sum of already-rounded items.

Where implemented: `labor_cost._rounded_cost`.

Why it exists: The two rounding strategies can disagree by a cent; per-line-item rounding is the one that matches how a real payroll statement itemizes cost.

Tests protecting it: `test_labor_cost_service.py`'s dedicated rounding-divergence test.

BR-009 — Account lockout after repeated failed logins

Description: 5 consecutive failed attempts lock an account for 15 minutes; a failure against an already-expired lock resets the counter instead of re-locking indefinitely.

Where implemented: `auth.service.authenticate`.

Why it exists: Bounds brute-force login attempts without permanently locking out a legitimate user who mistypes a password repeatedly.

Tests protecting it: Auth service integration tests covering lockout and reset behavior; `TestLoginTimingParity` (`test_auth_service_timing.py`) for the related timing defense.

BR-010 — Self-approval of leave is blocked

Description: An admin/manager who is also the requesting employee (via `scope.employee_id`) may not approve their own leave request.

Where implemented: `leave.approve_leave`.

Why it exists: A basic separation-of-duties control — an approval workflow with no independent reviewer is not a real approval workflow.

Tests protecting it: Leave service/route tests covering approval authorization.

BR-011 — Only an active employee may be newly scheduled, clocked in, or requested for leave

Description: An inactive or terminated employee cannot be assigned a new shift, cannot clock in fresh, and cannot have a new leave request filed for them — but their *existing* historical records are left untouched.

Where implemented: `scheduling._validate_employee_assignable`, `attendance._validate_employee_for_scope`, `leave._validate_employee_for_scope`.

Why it exists: A terminated employee must never appear on a future schedule, but rewriting their past schedule/attendance would falsify history.

Tests protecting it: Scheduling/attendance/leave service tests covering inactive/terminated employees.

BR-012 — Departments and organizations are deactivated, never deleted

Description: `deactivate_department` sets `is_active = false`; there is no delete path for a department, employee, or organization anywhere in the codebase.

Where implemented: `departments.deactivate_department`; reinforced at the schema level by `ON DELETE RESTRICT` on every foreign key referencing these tables (§3.5).

Why it exists: Historical schedules, attendance, and cost reports may still reference a department or employee long after it stops being active.

Tests protecting it: Department service/constraint tests.

7.5 Error Handling

Failure case	Where handled	Response
Invalid login	<code>auth.service.authenticate</code>	Generic "Invalid email or password.", flashed; identical for every failure reason.
Unauthorized (not logged in)	Flask-Login	Redirect to <code>/login</code> with a flashed message.
Forbidden (wrong role)	<code>role_required</code>	Plain 403 page.
Missing/out-of-scope employee, shift, leave request, etc.	every service's own scoped lookup	404 , never 403, so existence cannot be inferred (Chapter 2).
Invalid schedule (overlap, leave conflict)	<code>scheduling._check_overlap/_check_leave_conflict</code> , backed by DB exclusion constraints	A specific <code>ValidationError</code> message, flashed.
Invalid leave request (overlap, missing reason on rejection)	<code>leave.py</code>	Specific <code>ValidationError</code> , flashed.
Invalid attendance operation (duplicate clock-in, clock-out on a closed or needs-review entry)	<code>attendance.py</code> , backed by the DB's partial unique index	Specific <code>ValidationError</code> , flashed; never a raw <code>IntegrityError</code> .
Missing pay-rate/overtime-policy configuration	<code>labor_cost._daily_breakdown</code>	A named <code>ValidationError</code> ; department totals isolate the affected employee rather than failing the whole report (§4.6).

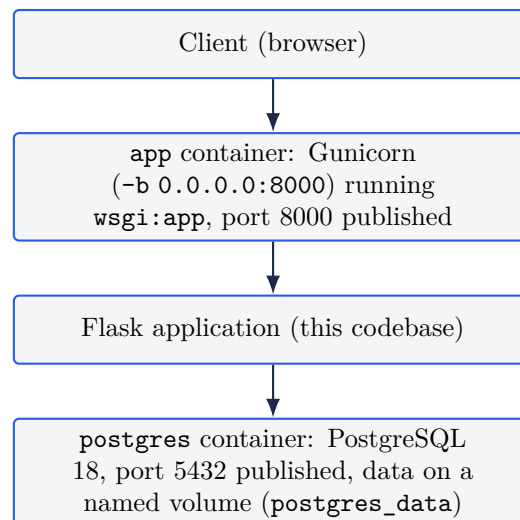
Failure case	Where handled	Response
Database constraint violation not otherwise translated	every <code>_commit_or_raise × _flush_or_raise*</code> helper	Re-raised as-is if the constraint name isn't recognized, ultimately caught by the route's generic <code>IntegrityError</code> handler and flashed as a conflict message.
Validation failure (form-level)	WTForms validators	Field-level error rendered next to the offending input.
Unhandled server error	<code>app.errors.handle_internal_server_error</code>	Generic 500 page; full exception logged server-side only.
Request body too large	Flask's <code>MAX_CONTENT_LENGTH</code>	413 page.

PART 8

Deployment, Maintenance, Limitations, and Roadmap

8.1 Deployment Architecture

The actual deployment configuration in the repository (`docker-compose.yml`, `Dockerfile`) defines exactly two containers:



No reverse proxy is present in this configuration. Gunicorn's port is published directly to the host. CLAUDE.md's technology stack section names "Nginx when appropriate" as conditional guidance, not a committed deployment component — the current `docker-compose.yml` does not include an Nginx (or any other reverse-proxy) service, so TLS termination, request buffering, and static-file serving in front of Gunicorn are all currently **[PLANNED / NOT IMPLEMENTED]** rather than configured. A real production deployment would need one of these added in front of the app service.

The Docker image itself (`Dockerfile`) is a straightforward `python:3.14-slim` base: dependencies installed from `requirements.txt`, the application copied in, and the process run as a dedicated non-root `app` user (`groupadd -system app; useradd -system ...`) — not root, a real hardening detail worth noting explicitly.

8.2 Configuration

Every configuration value is read from the environment (`app/config.py`), never hardcoded. The variables below are from `.env.example` (values redacted where they would normally be real secrets):

```

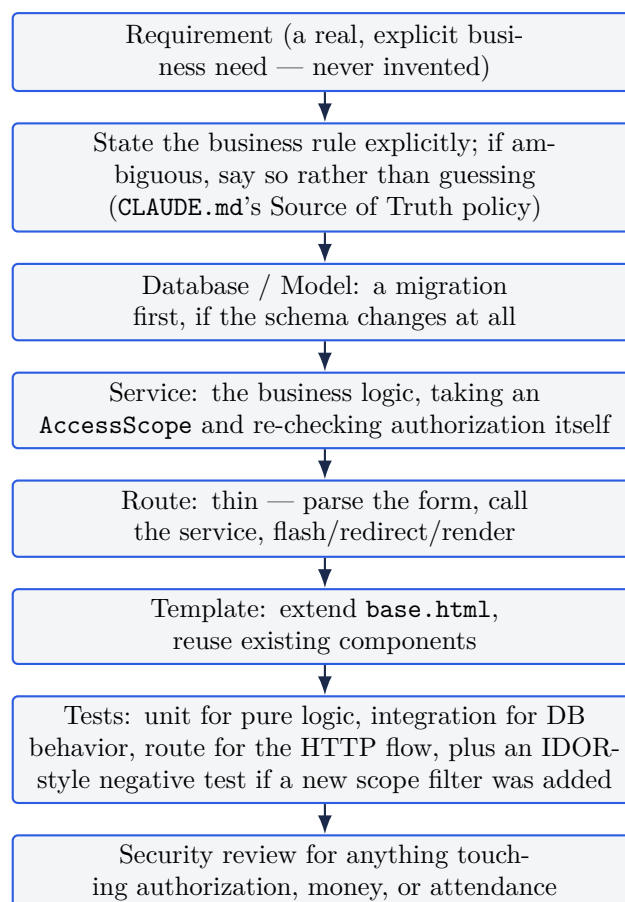
FLASK_APP=wsgi:app
FLASK_ENV=development # development | testing | production
SECRET_KEY=<REDACTED>
DATABASE_URL=<REDACTED> # postgresql+psycopg://user:pass@host:5432/db
TEST_DATABASE_URL=<REDACTED>

```

`docker-compose.yml` additionally expects `POSTGRES_USER`, `POSTGRES_PASSWORD`, and `POSTGRES_DB` for the `postgres` service, sourced from the same `.env` file (`env_file: .env` on the `app` service; the `postgres` service reads the same three variables directly from the shell environment when Compose is invoked).

`ProductionConfig` fails fast (`RuntimeError` at instantiation) if `SECRET_KEY` or `DATABASE_URL` is unset — the application cannot silently boot into an insecure default in that environment. `FLASK_ENV` **must** be set to `production` before any non-local deployment; an unset value defaults to `development` (`DEBUG=True`, non-secure cookies, no HSTS), which is appropriate for local development only.

8.3 How to Add a New Feature



What should *not* be done: a route querying the database directly; a business rule duplicated in two service functions instead of shared; a new one-off CSS class instead of an existing component; skipping the negative/scope test because "the happy path passes."

8.4 How to Add a New Page

1) Add a route function in the appropriate blueprint (or a new blueprint, registered in `create_app`). 2) Apply `login_required` or `role_required(...)` — never leave a route with no decorator by assuming a template will hide it appropriately. 3) Call an existing service function, or add a new one following the `AccessScope`-first, self-checking pattern used everywhere else. 4) Create a template extending `base.html`, reusing `icons.html/badges.html/existing` component classes rather than inventing new markup patterns. 5) Add CSS only if a genuinely new component is needed, as tokens-driven classes in `components.css` — never inline `style="..."` (the CSP blocks it) and never a page-specific stylesheet. 6) Add route-level tests covering the authorization boundary (who can/can't reach it) and at least one real data scenario, not just a 200-status smoke test.

8.5 How to Modify Existing Business Logic

Business logic lives in exactly one place per rule — find the owning service function (Chapter 4 documents where each domain's logic lives) and change it there, never by adding a special case in a route or template. Update the tests that exercise that function directly (both the "happy path" and any edge case already covered), and re-run the broader suite for that domain, not just the one changed test. Reconsider authorization explicitly: does the change affect what a manager vs. admin vs. employee can see or do? Reconsider database integrity: does an existing `CHECK/EXCLUDE` constraint still hold, or does the change require a new migration? Regression risk is highest exactly where this codebase has already been bitten once — overtime/labor-cost double-counting, timezone/business-date attribution, and IDOR-style scope leaks are the three areas with the most defensive test coverage for a reason.

8.6 Known Limitations

Area	Current state	Limitation	Impact	Future direction
Breaks	A single <code>break_minutes</code> integer per shift/entry	No break start/end events, no multiple breaks, no "on break" status	Product	Add a real break-session model if finer-grained tracking is required
Leave balances	<code>LeaveType</code> is a catalog only	No accrual/days-remaining ledger	Product	Add a balance ledger tied to leave type and employee
Paid leave costing	<code>LeaveType.is_paid</code> unused by labor cost	Approved leave contributes \$0 to every cost figure	Product / Financial accuracy	Requires a real business decision on OT interaction before implementing (§4.6)

Area	Current state	Limitation	Impact	Future direction
User provisioning	No self-service signup route exists	[NEEDS VERIFICATION] : provisioning path (CLI/direct DB) is not established in application code	Technical	Add an admin-facing "invite user" flow if needed
Rate limiting	Per-account lockout only (5 attempts / 15 min)	No IP-based or distributed rate limiting	Security	Add at the reverse proxy or application middleware layer
API	Server-rendered HTML only, no JSON API	No path for a future mobile app or external integration without one	Technical / Scalability	Add a versioned JSON API layer alongside existing routes if needed
Reverse proxy / TLS	Gunicorn's port published directly in <code>docker-compose.yml</code>	No Nginx or equivalent in the current deployment config	Technical	Add a reverse proxy in front of Gunicorn for any real deployment
Report range	Overtime/Hours-Trend/Labor-Cost queries clamped to 92 days	A genuine "since inception" report is not obtainable in one request	UX / Technical	Add pagination or background export for wider ranges if needed
Audit log immutability	Enforced by application convention only	No verified database-level protection (e.g. a restricted grant or trigger) against a future code path issuing UPDATE/DELETE	Security	[NEEDS VERIFICATION] and consider a database-level enforcement

8.7 Recommended Roadmap

MVP 1 (current)

The system as documented in this book is, by the project's own definition (CLAUDE.md), a complete MVP 1: authentication, roles, employees, departments, scheduling, attendance, leave, overtime, labor cost, reports, and audit logging are all implemented, tested, and authorization-reviewed.

MVP 2 (suggested, not committed)

Prioritized by the project's own stated order (security, data integrity, core workflows, UX, reporting, scalability, advanced functionality):

1. Resolve the two explicitly documented gaps in §8.6: paid-leave costing (needs a business decision first) and leave balances.
2. A real break-tracking model, if operationally needed.
3. A user-provisioning workflow (invite/reset flow) if account creation is currently a manual/CLI-only process in practice.
4. A reverse proxy and TLS termination for production deployment.

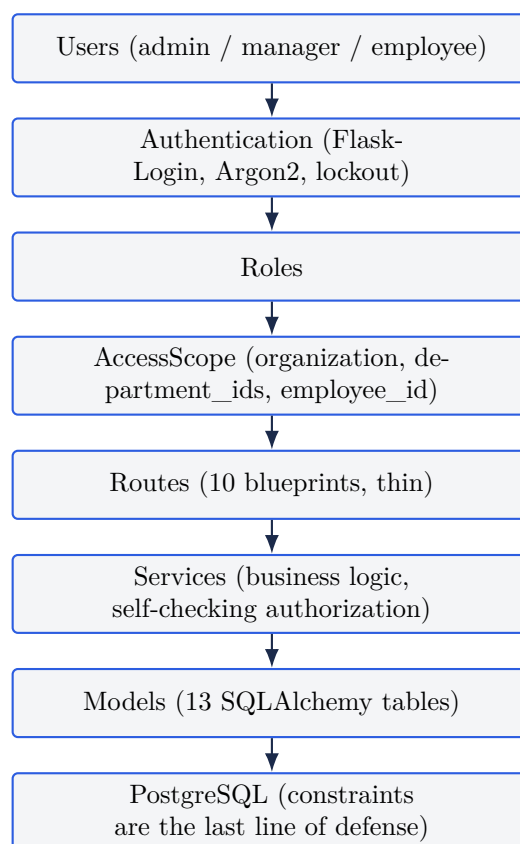
Future (vision, per CLAUDE.md)

Workforce demand forecasting, AI-assisted scheduling, and schedule optimization are named as the long-term direction for this product — explicitly *not* part of MVP 1 or MVP 2, and explicitly required (per CLAUDE.md) to consume this system’s verified data as an input, never to replace it as the authority. None of this exists in the codebase today; it is recorded here only because it is the project’s own stated direction, not because any of it is implemented or scheduled.

PART 9

Developer Quick Reference

9.1 One-Page System Map



Main domain relationships: Organization → Department → Employee → {Shift, AttendanceEntry, LeaveRequest, EmployeePayRate}; Employee ↔ User (0..1); User ↔ Department via DepartmentManager; OvertimePolicy → OvertimeTier; everything logs to AuditLog.

Core derived-value chain: Attendance (actual) + Schedule (planned) → Working Hours → Overtime → Labor Cost. Nothing in this chain is stored independently of the underlying attendance records — every figure is recomputed from them.

9.2 Important Files Cheat Sheet

File	Purpose		When I need to edit it
<code>app/auth/scope.py</code>	AccessScope <code>get_scoped_or_404</code>	+	Changing how authorization scope is computed — rare, high-risk.
<code>app/auth/decorators.py</code>	<code>login_required</code> <code>role_required</code>	/	Adding a new role or changing the coarse route-level check.
<code>app/auth/service.py</code>	Login/lockout logic	business	Changing password policy, lockout thresholds, or login audit behavior.
<code>app/models/base.py</code>	Naming convention <code>TimestampMixin</code>	+	Almost never — affects every table's generated constraint names.
<code>app/services/reports.py</code>	Read-only composition over every other service, for dashboards/reports		Adding a new dashboard metric or report — never invent a formula here, only reuse one.
<code>app/services/labor_cost.py</code>	Money calculations		Any change to how cost is priced or rounded — highest-risk file in the codebase; re-run <code>test_labor_cost_service.py</code> in full.
<code>app/services/overtime.py</code>	Pure overtime tiering math		Changing overtime rules — keep it database-free and unit-testable.
<code>app/services/attendance.py</code>	Clock-in/out, correction, stale-flagging		Any change to the attendance state machine.
<code>app/services/scheduling.py</code>	Shift lifecycle		Any change to shift creation, assignment, publish/cancel.
<code>app/forms.py</code>	Every WTForms class		Adding a new form field or a new form entirely.
<code>app/errors.py</code>	400/403/404/413/500 handlers		Adding a new handled status code — never leak exception detail to the response.
<code>app/__init__.py</code>	<code>create_app</code> , security headers, blueprint registration		Adding a new blueprint, or changing global headers/CSP.
<code>migrations/versions/</code>	Schema history		Any schema change — always via a new migration, never by hand-editing the database.
<code>templates/base.html</code>	The one shared page skeleton		Changing the application shell itself (rare).
<code>templates/components/*.html</code>	Shared macros/includes		Adding a new icon, a new badge variant, or changing nav structure.

File	Purpose	When I need to edit it
<code>static/css/tokens.css</code>	Every design token	Changing the accent color, spacing scale, or typography — one file, applies everywhere.
<code>tests/factories.py</code>	Test data builders	Adding a new model requires a matching factory here.
<code>tests/conftest.py</code>	<code>client/db_session</code> fixtures	Rarely — changes here affect every test in the suite.

9.3 Glossary

Blueprint

A Flask mechanism for grouping related routes under one prefix/namespace. This project has ten (attendance, audit, auth, dashboard, departments, employees, labor_cost, leave, main, schedule).

Route

A single URL pattern plus HTTP method, mapped to one view function.

Service

A module in `app/services/` containing business logic; the layer every route delegates to.

Model

A SQLAlchemy declarative class in `app/models/` representing one database table.

ORM

Object-Relational Mapper — SQLAlchemy’s mechanism for expressing database queries as Python objects/method calls instead of raw SQL.

AccessScope

The frozen dataclass (`app.auth.scope`) every authorization decision in the codebase is built from: `organization`, `role`, `department_ids`, `employee_id`.

Role

One of `admin`, `manager`, `employee` — stored on `User.role`, enforced by a database CHECK constraint.

Authentication

Confirming *who* is making a request (login, session cookie, Flask-Login).

Authorization

Confirming *what* an authenticated user is allowed to see or do (roles, AccessScope, scoped queries).

Attendance

Actual, recorded work (clock-in/clock-out) — distinct from a Shift.

Shift

Planned work — distinct from Attendance. Statuses: draft, published, cancelled.

Overtime

Extra pay owed once daily/weekly worked-hour thresholds are crossed, computed by configurable tiered multipliers.

Labor Cost

The Decimal monetary cost of worked hours, combining worked time, overtime tiering, and an employee's effective-dated hourly rate.

Audit Log

The append-only record of privileged actions (`audit_logs` table).

Migration

A versioned, ordered script (Alembic, via Flask-Migrate) that applies one schema change.

Transaction

A group of database operations committed or rolled back together — e.g. a primary write and its audit-log entry always share one commit in this codebase.

Constraint

A database-enforced rule (`CHECK`, `UNIQUE`, `FOREIGN KEY`, or `EXCLUDE`) that makes an invalid state physically unrepresentable, independent of application code.

Index

A database structure that speeds up a specific query pattern — added in this project only where a real, identified access pattern needed one.

9.4 Final System Summary

What the system is. A single-tenant-per-organization, server-rendered Flask application for managing an hourly workforce: employees, departments, schedules, attendance, leave, overtime, labor cost, reports, and an audit trail.

How it is structured. A modular monolith with a strict three-layer separation — thin Flask routes, self-authorizing service functions, and SQLAlchemy models backed by a PostgreSQL schema whose own constraints are a genuine second line of defense, not decoration.

How data flows. Browser → route → `AccessScope` → service (re-checks authorization, applies business rules) → SQLAlchemy → PostgreSQL, and back through a Jinja template to HTML — the same path for every page in the application, documented in full in Chapter 2.

How users interact with it. Through sixteen substantive pages (Chapter 5), all reachable from one consistent sidebar/topbar shell, each scoped to what the signed-in role is allowed to see.

How authentication works. Email/password with Argon2 hashing, generic failure messages, timing-attack mitigation, and a 5-attempt lockout — detailed in Chapter 2.

How authorization works. `AccessScope`, checked at up to three independent layers (route decorator, service function, database query/constraint), with a 404-not-403 IDOR defense throughout — detailed in Chapter 2 and threat-modeled in Chapter 6.

How the database is structured. Thirteen tables under a strict organization-scoped tenancy model, with five hand-written `EXCLUDE ... USING gist` constraints as the schema's signature integrity mechanism against every kind of double-booking the domain has (shifts, attendance, leave, pay rates, overtime policies) — detailed in Chapter 3.

What the most important business rules are. The twelve cataloged in Chapter 7, §7.4 — most centrally, that Schedule (planned) and Attendance (actual) are never conflated, that no worked hour is ever priced under two overtime multipliers, and that a manager never sees an individual's pay rate or exact cost.

What is currently implemented. Everything in the "Implemented" rows of §1.2's product scope table — which is to say, the complete MVP 1 as defined by the project's own governing document.

What remains to be built. Documented explicitly, not left implicit: leave balances, paid-leave costing, richer break tracking, and (for a real production deployment) a reverse proxy in front of Gunicorn — see Chapter 8, §8.6–8.7.

What a new developer needs to understand first. In order: the Employee/User distinction (§4.2) and the Schedule/Attendance distinction (§1.1, §4.3–4.4), since almost every other domain concept is built on top of those two separations; then `AccessScope` and the defense-in-depth authorization model (Chapter 2), since every service function assumes the reader already understands it; then `app/services/labor_cost.py` and `overtime.py` together, the two most intricate modules in the codebase, both extensively documented in their own docstrings and in Chapter 4, §4.5–4.6, of this book.